UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME TÉCNICO DE INVESTIGACIÓN FORMATIVA

"RUTAS MÍNIMAS CON DISTINTAS COLAS DE PRIORIDAD EN REDES VIALES URBANAS DE LA CIUDAD DEL CUSCO"

Asignatura: ALGORITMOS AVANZADOS (Semestre 2026-2)

Docente Responsable: Ing. Héctor Eduardo Ugarte Rojas

Identificador: GRUPO 5

Presentado por:

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

Cusco – Perú – 2026

Resumen

Este proyecto compara tres colas de prioridad dentro del mismo algoritmo de Dijkstra para resolver el problema de caminos mínimos desde un origen único en redes viales urbanas: un min-heap binario indexado, un heap 4-ario indexado y un Fibonacci heap. El caso aplicado utiliza una consulta sobre una red vial real de Cusco obtenida de OpenStreetMap (891 vértices y 2139 aristas dirigidas), mientras que la comparación controlada se realiza sobre instancias sintéticas reproducibles con perfiles disperso ($|E| = 4|V|$) y denso ($|E| = 0.08\,|V|(|V|-1)$). El protocolo mide tiempo con `perf_counter_ns`, desactiva el recolector de basura durante cada muestra, ejecuta 15 repeticiones sobre tres instancias por escala y reporta mediana e intervalo intercuartílico; la corrección se valida contra NetworkX y un certificado de optimalidad de Bellman en $O(|V|+|E|)$.

Los resultados no permiten declarar una cola ganadora universal. El heap 4-ario obtiene la menor mediana en 7 de 10 escalas sintéticas, con una razón mediana de 0.96 frente al heap binario en el perfil disperso y 0.95 en el denso, pero los intervalos intercuartílicos se solapan en las 10 escalas y el heap binario gana en 3 escalas y en la consulta OSM (15.65 ms frente a 19.87 ms del 4-ario y 26.09 ms del Fibonacci). El Fibonacci heap nunca obtiene la menor mediana y resulta entre 1.05 y 1.59 veces más lento que el binario. La extensión A^* reduce las expansiones de 887 a 823 ($-7.2\,\%$) sin alterar el costo óptimo. Un factor meteorológico uniforme de 1.0691 incrementa el tiempo estimado en 6.91 %, pero no modifica la ruta porque escala todos los pesos por igual. La conclusión central es que la elección de cola depende de la escala, la densidad y el entorno de ejecución, y que la ventaja asintótica del Fibonacci heap no se traduce en ventaja práctica en esta implementación de Python.

Palabras clave: algoritmo de Dijkstra, colas de prioridad, heap binario, heap 4-ario, Fibonacci heap, rutas mínimas, OpenStreetMap, Cusco, A^*, análisis experimental.

Abstract

This project compares three priority queues within the same Dijkstra algorithm for the single-source shortest path problem on urban road networks: an indexed binary min-heap, an indexed 4-ary heap, and a Fibonacci heap. The applied case uses a real road query in Cusco retrieved from OpenStreetMap (891 vertices and 2139 directed edges), while the controlled comparison uses reproducible synthetic instances with sparse ($|E| = 4|V|$) and dense ($|E| = 0.08\,|V|(|V|-1)$) profiles. The protocol measures wall-clock time with `perf_counter_ns`, disables garbage collection during each sample, runs 15 repetitions on three instances per scale, and reports the median and interquartile range; correctness is validated against NetworkX and a Bellman optimality certificate in $O(|V|+|E|)$.

The results do not support a universal winner. The 4-ary heap obtains the lowest median in 7 of 10 synthetic scales, with a median ratio of 0.96 relative to the binary heap in the sparse profile and 0.95 in the dense profile, but the interquartile ranges overlap in all 10 scales; the binary heap wins in 3 scales and in the OSM query (15.65 ms versus 19.87 ms for the 4-ary heap and 26.09 ms for the Fibonacci heap). The Fibonacci heap never obtains the lowest median and is 1.05 to 1.59 times slower than the binary heap.

The A^* extension reduces expansions from 887 to 823 ($-7.2\,\%$) without changing the optimal cost. A uniform weather factor of 1.0691 increases the estimated time by $6.91\,\%$ but does not change the route because it scales every weight equally. The central conclusion is that queue choice depends on scale, density, and runtime environment, and that the asymptotic advantage of the Fibonacci heap does not translate into a practical advantage in this Python implementation.

Keywords: Dijkstra's algorithm, priority queues, binary heap, 4-ary heap, Fibonacci heap, shortest paths, OpenStreetMap, Cusco, A^*, experimental analysis.

Índice

Resumen I

Abstract I

2. Introducción 1
- 2.1. Contexto y motivación . . . 1
- 2.2. Planteamiento del problema . . . 1
- 2.3. Objetivos . . . 1
- 2.4. Alcance y delimitaciones . . . 2
- 2.5. Organización del documento . . . 2
- 2.6. Repositorio y control de versiones . . . 2

3. Fundamentos Teóricos y Aprendizaje Autónomo 3
- 3.1. Conceptos y algoritmos requeridos . . . 3
- 3.2. Fuentes estudiadas y criterios de selección . . . 4
- 3.3. Brechas de conocimiento y estrategia de aprendizaje . . . 4
- 3.4. Evidencias de aprendizaje autónomo . . . 5

4. Evaluación y Selección del Stack Tecnológico 6
- 4.1. Alternativas consideradas . . . 6
- 4.2. Criterios de comparación . . . 6
- 4.3. Tecnología seleccionada . . . 6
- 4.4. Procedimiento de aprendizaje e integración . . . 7
- 4.5. Dificultades de adopción y soluciones . . . 7

5. Formulación Matemática del Problema y Caso de Estudio 8
- 5.1. Definición formal . . . 8
- 5.2. Modelo de costo temporal . . . 8
- 5.3. Instancia ilustrativa de seis nodos . . . 9
- 5.4. Traza manual de Dijkstra . . . 10
- 5.5. Instancia aplicada: red OSM de Cusco . . . 11

6. Diseño Algorítmico y Estructuras de Cola 12
- 6.1. Algoritmo de Dijkstra parametrizado . . . 12
- 6.2. Invariante y corrección . . . 12
- 6.3. Heap binario indexado . . . 13
- 6.4. Heap 4-ario indexado . . . 13
- 6.5. Fibonacci heap . . . 13
- 6.6. Complejidades comparadas . . . 14
- 6.7. Criterios de selección . . . 14
- 6.8. Correspondencia entre pseudocódigo y código . . . 14

7. Diseño e Implementación del Sistema 16
- 7.1. Arquitectura modular . . . 16

7.2. Módulos principales 16
7.3. Formatos de datos 17
7.4. Contratos e inyección de colas 18
7.5. Manejo defensivo de entradas 18
7.6. Decisiones técnicas 18

8. Estrategia de Pruebas y Validación Automatizada 19
8.1. Objetivo y organización 19
8.2. Patrón Arrange-Act-Assert 20
8.3. Pruebas del contrato de colas 20
8.4. Oráculos de caminos mínimos 20
8.5. Certificado de optimalidad 21
8.6. Pruebas de borde y regresión 21
8.7. Pruebas de integración sin red 21
8.8. Criterio de aceptación 22

9. Metodología Experimental y Diseño del Banco de Pruebas 23
9.1. Preguntas de investigación 23
9.2. Hipótesis de trabajo 23
9.3. Diseño factorial 23
9.4. Generación de instancias 24
9.5. Protocolo de medición 24
9.6. Caso aplicado sobre OpenStreetMap 25
9.7. Entorno de ejecución 25
9.8. Reproducibilidad 26
9.9. Amenazas a la validez 26

10. Resultados Computacionales y Evaluación Experimental 28
10.1. Verificación de corrección 28
10.2. Benchmark sintético 28
10.3. Operaciones elementales 30
10.4. Caso aplicado: consulta OSM en Cusco 31
10.5. Sensibilidad meteorológica 32

11. Discusión Crítica de Hallazgos 33
11.1. Teoría asintótica y tiempo de pared 33
11.2. Por qué el heap 4-ario gana en algunas escalas 33
11.3. Por qué el heap binario gana en la consulta OSM 34
11.4. El papel de la densidad 34
11.5. Extensión A^* 34
11.6. Sensibilidad climática 34
11.7. Marco de decisión 35
11.8. Limitaciones 35
11.9. Validez externa 35

12. Conclusiones y Recomendaciones **36**
12.1. Balance de objetivos 36
12.2. Respuesta a las preguntas de investigación 36
12.3. Conclusión principal 37
12.4. Contribuciones 37
12.5. Recomendaciones 37
12.6. Cierre 38

Referencias Bibliográficas **39**

Anexos **40**
Anexo A: Instalación y configuración 40
Anexo B: Uso del prototipo 40
Anexo C: Bitácora de aprendizaje 41
Anexo D: Matriz de trazabilidad 42
Anexo E: Matriz de contribuciones 43
Anexo F: Evidencias de seguimiento 43

Índice de figuras

1. Instancia ilustrativa del Centro Histórico ($|V| = 6$, $|E| = 9$). Las etiquetas son longitudes esquemáticas en metros. . . . 9
2. Arquitectura del sistema en cuatro capas . . . 16
3. Tiempo mediano de Dijkstra con las tres colas. Izquierda: perfil disperso ($|E| = 4|V|$). Derecha: perfil denso ($|E| = 0.08|V|(|V| - 1)$). Las barras verticales representan el IQR. 29
4. Operaciones primitivas de Dijkstra en el perfil disperso. Las barras agrupan inserciones, extracciones, `decrease_key` efectivos e intentos de relajación. . . . 30
5. Ruta mínima sobre la red vial OSM de Cusco. La línea resalta el camino óptimo; los marcadores indican origen y destino. . . . 31

Índice de tablas

1. Fuentes principales y criterio de pertinencia . . . 4
2. Plan de aprendizaje y responsabilidades . . . 5
3. Alternativas por capa tecnológica . . . 6
4. Matriz multicriterio del lenguaje principal (escala 1–5) . . . 6
5. Velocidades nominales por tipo de vía . . . 9
6. Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco. . . . 10
7. Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo. . . . 11
8. Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor 11
9. Cotas temporales y espaciales de las colas evaluadas . . . 14
10. Guía de selección según el escenario . . . 14
11. Correspondencia entre pseudocódigo y código . . . 15
12. Archivos de datos y su función . . . 17
13. Distribución de la suite automatizada . . . 19
14. Entorno de ejecución del benchmark . . . 26
15. Dijkstra con tres colas, perfil disperso: mediana e IQR en μs. . . . 28
16. Dijkstra con tres colas, perfil denso: mediana e IQR en μs. . . . 29
17. Síntesis comparativa por perfil y consulta. Las razones usan la mediana; no se declara un ganador cuando los IQR se solapan. . . . 30
18. Caso OSM: tiempo de Dijkstra por cola, mediana e IQR. . . . 32
19. Sensibilidad meteorológica uniforme en la consulta OSM. . . . 32
20. Bitácora de aprendizaje y resolución de dificultades . . . 42
21. Trazabilidad entre objetivos, componentes y evidencia . . . 43
22. Contribuciones por integrante y evidencia en el repositorio . . . 43

2. Introducción

2.1. Contexto y motivación

El cálculo de rutas mínimas en una red vial es un problema central en logística, navegación y atención de emergencias. En una ciudad con topografía irregular, la ruta más corta en distancia no es necesariamente la más rápida: una pendiente pronunciada, una calle estrecha o un sentido único pueden aumentar el tiempo de viaje. Por ello, el problema se modela como un grafo dirigido con pesos no negativos que representan tiempo estimado, y se resuelve con el algoritmo de Dijkstra (Dijkstra, 1959).

La literatura demuestra que el costo de Dijkstra depende de la cola de prioridad utilizada. El Fibonacci heap de Fredman y Tarjan (1987) alcanza la cota asintótica $O(|E|+|V|\log|V|)$, mientras que un heap binario indexado requiere $O((|V|+|E|)\log|V|)$. Sin embargo, la cota asintótica oculta constantes de implementación, localidad de memoria y costos de asignación dinámica. Cherkassky et al. (1996) y LaMarca y Ladner (1996) muestran que, en redes dispersas y en implementaciones reales, los heaps contiguos suelen superar al Fibonacci heap. Este proyecto traslada esa discusión a una red vial de Cusco y la somete a un protocolo reproducible.

2.2. Planteamiento del problema

Sea $G=(V,E,w)$ un grafo dirigido con $w(e)\geq 0$. Para un origen $s\in V$, se busca un vector $d:V\to\mathbb{R}_{\geq 0}\cup\{\infty\}$ tal que $d(v)=\delta(s,v)$, donde $\delta(s,v)$ es el costo mínimo de un camino de s a v. Dijkstra mantiene una cola de prioridad Q con las distancias tentativas y ejecuta tres operaciones: `Insert`, `Extract-Min` y `Decrease-Key`. Su costo total es

$$T(|V|,|E|)=O\big(|V|\,T_{\text{Extract-Min}}+|E|\,T_{\text{Decrease-Key}}+|V|\,T_{\text{Insert}}\big). \tag{1}$$

La pregunta central es empírica: *¿qué cola conviene en una red vial urbana, bajo qué escala y densidad, y por qué?* La respuesta no se presume; se mide con un benchmark canónico y se contrasta con la teoría.

2.3. Objetivos

Objetivo general. Diseñar, implementar y evaluar experimentalmente una solución modular de rutas mínimas para una red vial urbana de Cusco, comparando el algoritmo de Dijkstra con tres colas de prioridad y verificando la corrección de los resultados.

Objetivos específicos.

1. **OE1. Modelar:** formular el grafo vial, la función de costo temporal y las restricciones de no negatividad.
2. **OE2. Implementar:** construir desde cero el heap binario indexado, el heap 4-ario indexado y el Fibonacci heap, con una interfaz común que permita sustituirlos dentro de Dijkstra.
3. **OE3. Validar:** comprobar las distancias con una traza manual, con NetworkX y con un certificado de optimalidad de Bellman en $O(|V|+|E|)$, además de una suite automatizada de pruebas.

4. **OE4. Evaluar:** medir tiempo y operaciones elementales en instancias sintéticas y en una consulta real de OpenStreetMap, y analizar la extensión A^* y la sensibilidad meteorológica.

2.4. Alcance y delimitaciones

- **Red de estudio.** La comparación controlada usa grafos sintéticos con semilla fija. El caso aplicado usa una consulta sobre un recorte vial real de Cusco de 1.5 km de radio, con 891 vértices y 2139 aristas dirigidas, obtenido con OSMnx (Boeing, 2017). La instancia de seis nodos es ilustrativa: los nombres de las plazas son reales, pero longitudes, pendientes y topología son un ejemplo docente, no un levantamiento geográfico.
- **Modelo de costo.** Los pesos son deterministas y no negativos. No se modela congestión en tiempo real ni variación horaria. El clima se incorpora como un factor uniforme de escenario, separado del benchmark principal.
- **Alcance algorítmico.** Dijkstra es el objeto principal; A^* se incluye como extensión punto a punto con heurística consistente. Bellman-Ford y NetworkX se usan como oráculos, no como alternativas del benchmark.
- **Implementación.** Python 3.12, sin usar `heapq` ni colas de bibliotecas para las estructuras evaluadas. El objetivo es comparar las implementaciones del proyecto, no las de una biblioteca optimizada en C.

2.5. Organización del documento

La Sección 3 presenta los fundamentos y el aprendizaje autónomo (AG_C06.01). La Sección 4 justifica el stack tecnológico (AG_C06.02). La Sección 5 formula el problema y resuelve la instancia ilustrativa. La Sección 6 describe los algoritmos, las colas, los invariantes y las complejidades. La Sección 7 documenta la arquitectura y los formatos. La Sección 8 describe la estrategia de pruebas. La Sección 9 define el protocolo experimental (AG_C06.03). La Sección 10 presenta los resultados. La Sección 11 los interpreta y delimita. La Sección 12 responde a los objetivos. Finalmente se incluyen las referencias y los anexos técnicos.

2.6. Repositorio y control de versiones

El código, los datos procesados y el informe se mantienen en el repositorio público del grupo:

https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC

La rama `main` contiene la versión estable; las ramas temáticas documentan el trabajo por integrante. Los datos experimentales se versionan en `python/data/processed/` para que las tablas y figuras puedan regenerarse sin depender de una descarga en vivo.

3. Fundamentos Teóricos y Aprendizaje Autónomo

3.1. Conceptos y algoritmos requeridos

Una red vial urbana se modela como un grafo dirigido ponderado $G = (V, E, w)$, donde V son intersecciones o cambios de geometría, $E \subseteq V \times V$ son tramos con sentido de circulación y $w : E \to \mathbb{R}_{\geq 0}$ es el costo. Las redes viales son dispersas: en el recorte OSM utilizado, $|E|/|V| = 2139/891 = 2.40$. Esta razón condiciona el peso relativo de las operaciones de cola.

Dijkstra e invariante. Dijkstra mantiene un conjunto S de vértices con distancia definitiva y una cola $Q = V \setminus S$ con distancias tentativas. En cada iteración extrae $u = \arg\min_{z \in Q} d[z]$, lo incorpora a S y relaja las aristas salientes:

$$\text{si } d[u] + w(u,v) < d[v], \quad d[v] \leftarrow d[u] + w(u,v),\ \pi[v] \leftarrow u. \tag{2}$$

El invariante es: al inicio de cada iteración, $d[u] = \delta(s,u)$ para todo $u \in S$, y $d[v]$ es el costo mínimo de un camino de s a v con vértices intermedios en S. La demostración es por inducción. Si existiera un camino más corto hacia el vértice extraído u, ese camino abandonaría S por una arista (x,y) con $x \in S$ e $y \in Q$; entonces

$$w(P) \geq \delta(s,x) + w(x,y) = d[y] \geq d[u], \tag{3}$$

lo que contradice $w(P) < d[u]$. La no negatividad de los pesos es esencial para la última desigualdad.

Colas de prioridad. Las tres estructuras evaluadas implementan el mismo contrato: `insert`, `extract_min`, `decrease_key` e `is_empty`. El heap binario indexado usa un arreglo contiguo y una tabla pos : $V \to \mathbb{N}$ para localizar cada vértice en $O(1)$. El heap 4-ario generaliza la aritmética de índices:

$$\text{padre}(i) = \left\lfloor \frac{i-1}{4} \right\rfloor, \qquad \text{hijo}_k(i) = 4i + k, \quad k = 1, \ldots, 4. \tag{4}$$

El Fibonacci heap usa una colección de árboles enlazados, inserción perezosa, cortes en cascada y consolidación durante `extract_min` (Fredman & Tarjan, 1987). La Tabla 9 resume las cotas.

Búsqueda A^*. Para una consulta punto a punto $s \to t$, A^* ordena por $f(v) = g(v) + h(v)$, donde g es el costo acumulado y h una estimación del costo restante (Hart et al., 1968). Si h es admisible, $h(v) \leq h^*(v)$, el algoritmo es óptimo. Si además es consistente,

$$h(u) \leq w(u,v) + h(v) \quad \forall (u,v) \in E, \tag{5}$$

entonces f no decrece a lo largo de un camino y no es necesario reabrir vértices cerrados. En este proyecto, cuando la instancia tiene coordenadas, se usa

$$h(v) = \lambda \, \|x_v - x_t\|_2, \qquad \lambda = \min_{(u,v) \in E,\ \|x_u - x_v\| > 0} \frac{w(u,v)}{\|x_u - x_v\|}. \tag{6}$$

Por la desigualdad triangular, $h(u) \leq \lambda \|x_u - x_v\| + h(v) \leq w(u,v) + h(v)$; la consistencia se verifica antes de ejecutar el algoritmo. Si faltan coordenadas, se usa $h = 0$ y A^* se reduce a Dijkstra. Esta formulación evita suponer una velocidad máxima válida para cualquier grafo.

3.2. Fuentes estudiadas y criterios de selección

La Tabla 1 resume las fuentes principales. Se priorizaron artículos seminales, textos de referencia y estudios experimentales con metodología explícita. Las afirmaciones sobre rendimiento se contrastan con el benchmark propio; las fuentes se usan para justificar el diseño y para interpretar, no para sustituir, la evidencia local.

Tabla 1
Fuentes principales y criterio de pertinencia

Fuente	Tipo y dominio	Uso en el proyecto
Dijkstra (1959)	Artículo seminal, algoritmos	Definición del SSSP y de la relajación voraz.
Fredman & Tarjan (1987)	Artículo teórico, estructuras	Cotas amortizadas del Fibonacci heap y operaciones.
Tarjan (1983)	Monografía, análisis amortizado	Método del potencial y análisis de costos.
Cherkassky et al. (1996)	Estudio experimental, caminos mínimos	Contraste entre teoría y rendimiento práctico.
LaMarca & Ladner (1996)	Artículo, arquitectura y algoritmos	Efecto de la caché en heaps binarios y d-arios.
Boeing (2017)	Artículo, sistemas geoespaciales	Descarga, proyección y conversión de redes OSM.
Cormen et al. (2022)	Libro de texto, algoritmos	Pseudocódigo, invariantes y demostraciones.
Hart et al. (1968)	Artículo seminal, búsqueda heurística	Admisibilidad y consistencia de A^*.

3.3. Brechas de conocimiento y estrategia de aprendizaje

Al inicio del proyecto el equipo identificó cuatro brechas:

1. **Estructuras enlazadas.** Implementar listas circulares, punteros padre/hijo/izquierda/derecha y cortes en cascada sin ciclos ni fugas lógicas.

2. **Indexación inversa.** Mantener sincronizada la posición de cada vértice durante `sift-up` y `sift-down`, para que `decrease_key` no degrade a $O(|V|)$.

3. **Geometría y datos OSM.** Descargar, proyectar y convertir una red vial real, preservando direccionalidad y atributos de velocidad.

4. **Medición de tiempos.** Aislar ruido del sistema operativo, calentar la caché y reportar una estadística robusta en lugar de promedios simples.

Tabla 2
Plan de aprendizaje y responsabilidades

Integrante	**Línea principal**	**Evidencia de aprendizaje**
Choquenaira, Noe	Arquitectura, modelado y A^*	Diseño de capas, función de costo, heurística consistente y coordinación del repositorio.
Porroa, Yeni	Heaps contiguos indexados	Implementación del heap binario y 4-ario, tabla de posiciones y análisis de localidad.
Quispe, Romario	Fibonacci heap y benchmarking	Cortes en cascada, consolidación, generador de instancias y protocolo de medición.
Yaranga, Aldo	Pruebas, CLI y documentación	Suite AAA, certificado de Bellman, interfaz de comandos y exportación LaTeX.

Dificultades resueltas. (i) La desincronización de la tabla de posiciones se resolvió encapsulando el intercambio en una operación atómica que actualiza el arreglo y el mapa inverso. (ii) Los cortes en cascada del Fibonacci heap se validaron con pruebas de invariantes y con un oráculo de distancias. (iii) La instalación de OSMnx en Windows se aisló mediante un *snapshot* JSON versionado, de modo que las pruebas y la aplicación funcionan sin red. (iv) La codificación de consola en Windows se estandarizó a UTF-8 y a mensajes ASCII en los flujos de registro.

3.4. Evidencias de aprendizaje autónomo

El aprendizaje se documenta en el repositorio (commits, pruebas y datos), en la bitácora del Anexo C y en la matriz de contribuciones del Anexo E. Cada integrante implementó al menos un componente verificable y participó en la revisión de otro. La defensa individual se apoya en la capacidad de ejecutar, modificar y explicar cualquiera de los módulos, no en una división rígida de tareas.

4. Evaluación y Selección del Stack Tecnológico

4.1. Alternativas consideradas

La Tabla 3 resume las alternativas evaluadas en cinco capas. No se valoró la novedad por sí misma, sino la capacidad de aprender, integrar, probar y justificar cada herramienta.

Tabla 3
Alternativas por capa tecnológica

Capa	Alternativas	Criterio principal
Lenguaje	C++20, Rust 1.75, Python 3.12	Equilibrio entre control de bajo nivel, expresividad y velocidad de experimentación.
Red vial	OSMnx/NetworkX, Overpass API, PostGIS	Descarga reproducible, conversión a grafo dirigido y atributos de velocidad.
Pruebas	pytest, unittest, GoogleTest	Parametrización, fixtures, aserciones y integración con Python.
Interfaz	Streamlit, Dash, Flask + HTML	Costo de desarrollo, accesibilidad y despliegue local reproducible.
Documentación	LaTeX, Typst, Word	Ecuaciones, tablas, referencias APA y control de versiones.

4.2. Criterios de comparación

Se usaron cuatro criterios ponderados: C1 expresividad algorítmica (30 %), C2 instrumentación y aislamiento del ruido (25 %), C3 reproducibilidad y portabilidad (25 %) y C4 curva de aprendizaje (20 %). La Tabla 4 presenta una escala ordinal de 1 a 5. Los puntajes son un juicio técnico del equipo, no una medición física; se incluyen para hacer explícita la decisión, no para simular precisión inexistente.

Tabla 4
Matriz multicriterio del lenguaje principal (escala 1–5)

Alternativa	C1 (30 %)	C2 (25 %)	C3 (25 %)	C4 (20 %)	Ponderado
Python 3.12	5.0	4.2	4.8	4.8	**4.71**
C++20	3.8	4.8	3.5	2.8	3.78
Rust 1.75	4.0	4.6	3.6	2.2	3.69

4.3. Tecnología seleccionada

1. **Python 3.12.8.** Permite expresar los algoritmos con tipado y pruebas sin una cadena de compilación. Las tres colas se implementaron desde cero; no se usó `heapq` para las estructuras evaluadas. La contrapartida es que los tiempos absolutos incluyen la sobrecarga del intérprete y no son extrapolables a C++ o Rust.

2. **OSMnx 2.1.1 y NetworkX 3.7.** OSMnx descarga y convierte la red vial; NetworkX se usa como

oráculo de distancias. La red se congela en JSON para que el análisis no dependa de una descarga en vivo.

3. **pytest 9.1.1.** Pruebas parametrizadas, fixtures y ejecución selectiva. La suite completa se ejecuta con `python -m pytest python/tests -q`.

4. **Streamlit 1.64.0 y Plotly 7.1.0.** Una sola aplicación web para comparar resultados, explorar la ruta OSM, revisar la traza manual y consultar la procedencia de los datos. La interfaz es un medio de interacción; el núcleo evaluado sigue siendo el algoritmo.

5. **LaTeX con BibLaTeX y Biber.** Informe reproducible, ecuaciones y referencias APA. La compilación se automatiza con `compilar.ps1`.

6. **Open-Meteo.** Fuente meteorológica inicial, identificada como datos de modelo. Cada consulta se guarda en un *snapshot* JSON con *fallback* local; no se atribuye a una estación física ni a SENAMHI.

4.4. Procedimiento de aprendizaje e integración

La adopción se organizó en tres fases. Primero se construyó un prototipo mínimo con el heap binario y la instancia de seis nodos. Después se añadieron el heap 4-ario, el Fibonacci heap y el certificado de optimalidad. Finalmente se integraron OSMnx, el benchmark canónico, la aplicación Streamlit y el servicio meteorológico con caché. Cada fase dejó pruebas y datos versionados; las decisiones que cambiaron el diseño se registraron en el repositorio.

4.5. Dificultades de adopción y soluciones

- **Dependencias geoespaciales en Windows.** La instalación de GDAL y OSMnx puede fallar por binarios incompatibles. Se resolvió fijando versiones en `requirements.txt` y exportando la red a JSON; las pruebas usan el *snapshot*, no la descarga.

- **Codificación de consola.** La consola CP1252 producía errores con caracteres Unicode. Se forzó UTF-8 en archivos y se usaron mensajes ASCII en los registros de la CLI.

- **Reproducibilidad temporal.** Las primeras mediciones usaban promedios y no aislaban el recolector de basura. Se adoptó `perf_counter_ns`, *warm-up*, `gc.disable()` durante la muestra y mediana/IQR.

- **Interfaz y estado.** Streamlit reejecuta el script en cada interacción. Se separaron las funciones puras de cálculo y se cachearon la carga de datos y la consulta meteorológica.

- **Clima sin API oficial de SENAMHI.** No se encontró una API pública documentada de SENAMHI. Se usó Open-Meteo como datos de modelo y se declaró esa limitación; el sistema conserva un *snapshot* para funcionar sin red.

5. Formulación Matemática del Problema y Caso de Estudio

5.1. Definición formal

Sea $G = (V, E, w)$ un grafo dirigido y ponderado. V es el conjunto de intersecciones, $E \subseteq V \times V$ el conjunto de tramos orientados según el sentido de circulación y $w : E \to \mathbb{R}_{\geq 0}$ el costo de cada tramo. Para un origen $s \in V$ y un destino $t \in V$, se busca

$$\delta(s,t) = \min_{P \in \mathscr{P}(s,t)} \sum_{e \in P} w(e), \tag{7}$$

donde $\mathscr{P}(s,t)$ es el conjunto de caminos dirigidos de s a t. Las variables de decisión son binarias:

$$x_{uv} = \begin{cases} 1, & \text{si } (u,v) \text{ pertenece al camino óptimo,} \\ 0, & \text{en caso contrario.} \end{cases} \tag{8}$$

El problema puede escribirse como el programa entero

$$\min Z = \sum_{(u,v) \in E} w(u,v)\, x_{uv} \tag{9}$$

sujeto a conservación de flujo unitario,

$$\sum_{v:(u,v) \in E} x_{uv} - \sum_{v:(v,u) \in E} x_{vu} = \begin{cases} 1, & u = s, \\ -1, & u = t, \\ 0, & u \in V \setminus \{s,t\}, \end{cases} \tag{10}$$

y a $x_{uv} \in \{0,1\}$, $w(u,v) \geq 0$. La no negatividad garantiza que Dijkstra sea aplicable; los ciclos de costo positivo nunca forman parte de un camino óptimo.

5.2. Modelo de costo temporal

El costo aplicado es el tiempo estimado de viaje en segundos:

$$w(u,v) = \frac{L(u,v)}{v_{\text{nom}}(\tau(u,v))} \cdot \Big(1 + \alpha \max\big(0, \theta(u,v)\big)\Big) \cdot m_{\text{clima}}, \tag{11}$$

donde L es la longitud en metros, τ el tipo de vía, θ la pendiente como fracción (por ejemplo, 0.03 para 3 %), $\alpha = 2.5$ y $m_{\text{clima}} \geq 1$ un factor uniforme de escenario. Las velocidades nominales se presentan en la Tabla 5. El término de pendiente solo penaliza ascensos; las bajadas conservan la velocidad nominal. Esta elección es un supuesto de escenario, no una calibración de tránsito.

Tabla 5
Velocidades nominales por tipo de vía

Tipo de vía	Velocidad (km/h)	Velocidad (m/s)
Avenida	35	9.72
Calle residencial	25	6.94
Calle de centro histórico	15	4.17
Peatonal restringida	8	2.22

Supuestos. (i) Los pesos son deterministas y no negativos. (ii) La velocidad nominal representa una condición de flujo libre; no se modela congestión horaria. (iii) La pendiente se aplica solo en ascenso y de forma lineal. (iv) El factor climático es uniforme y no depende del tramo; por tanto, no puede cambiar el orden de los caminos y se usa como análisis de sensibilidad, no como modelo de tráfico. (v) La instancia de seis nodos es ilustrativa; la red OSM es el caso aplicado real.

5.3. Instancia ilustrativa de seis nodos

La Figura 1 y la Tabla 6 definen una instancia docente de $|V| = 6$ y $|E| = 9$. Los nombres de las plazas son reales, pero las longitudes, pendientes y conexiones son esquemáticas: sirven para verificar la traza y la reconstrucción de rutas, no para afirmar una distancia geográfica exacta.

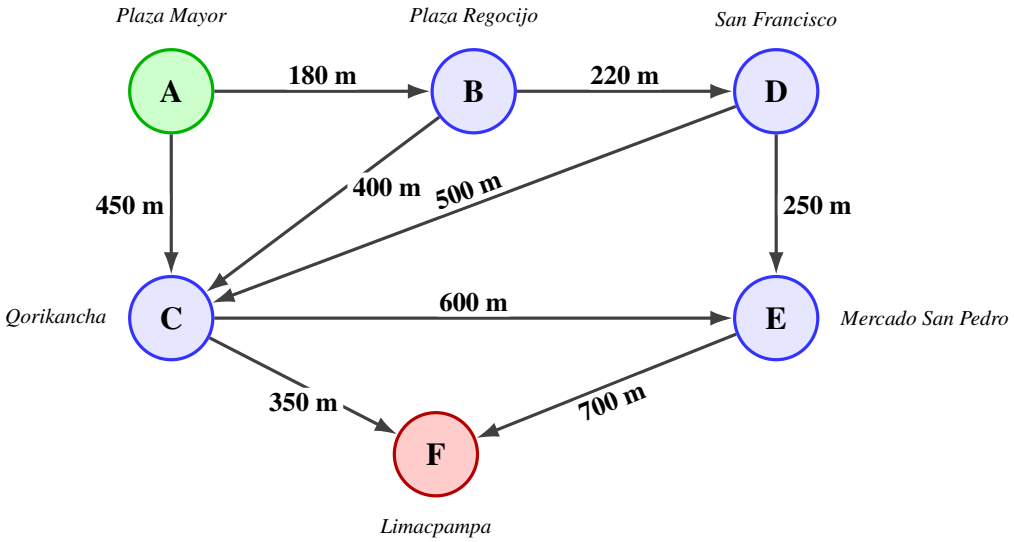


Figura 1
Instancia ilustrativa del Centro Histórico ($|V| = 6$, $|E| = 9$). Las etiquetas son longitudes esquemáticas en metros.

Tabla 6
Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco.

Origen	Destino	Vía Urbana Asociada	Distancia (m)	Pendiente (%)	Tiempo Est. (s)
A	B	Calle Espaderos / Portal Carnicerías	180	+1.5	44.8
A	C	Calle Loreto / Pampa del Castillo	450	-2.0	108.0
B	D	Calle Garcilaso / Calle Marqués	220	+3.0	56.8
B	C	Calle San Bernardo / Almagro	400	-1.0	96.0
C	F	Calle Ahuacpinta / Limacpampa Grande	350	+0.5	36.5
C	E	Calle Matará / Tres Cruces de Oro	600	+2.0	151.2
D	E	Calle Santa Clara / Arco Santa Clara	250	+2.5	63.8
D	C	Calle Marqués / Mantas	500	-2.5	120.0
E	F	Calle Nueva Baja / Av. Tullumayo	700	-1.0	72.0

Nota. Tiempo de viaje estimado según la función de costo e impedancia por pendiente andina ($\alpha = 2.5$).

5.4. Traza manual de Dijkstra

Se usa la métrica de distancia para que la traza sea verificable a mano. La Tabla 7 resume la evolución; el origen es $s = \text{A}$.

- $k = 0$**.** $d[A] = 0$, $d[v] = \infty$ para $v \neq A$, $\pi[v] = \text{NIL}$, $S = \emptyset$.
- $k = 1$**.** Se extrae A ($d = 0$). Se relajan (A,B) y (A,C): $d[B] = 180$, $\pi[B] = A$; $d[C] = 450$, $\pi[C] = A$.
- $k = 2$**.** Se extrae B ($d = 180$). (B,D) produce $d[D] = 400$, $\pi[D] = B$. (B,C) produce 580, que no mejora $d[C] = 450$.
- $k = 3$**.** Se extrae D ($d = 400$). (D,E) produce $d[E] = 650$, $\pi[E] = D$. (D,C) produce 900, que no mejora C.
- $k = 4$**.** Se extrae C ($d = 450$). (C,F) produce $d[F] = 800$, $\pi[F] = C$. (C,E) produce 1050, que no mejora E.
- $k = 5$**.** Se extrae E ($d = 650$). (E,F) produce 1350, que no mejora F.
- $k = 6$**.** Se extrae F ($d = 800$). $Q = \emptyset$; el algoritmo termina.

Tabla 7
Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo.

Iter.	Vértice u	Acción / Relajación	A	B	C	D	E	F
$k=0$	Inicial	Inserción en cola	0 / -	∞ / -	∞ / -	∞ / -	∞ / -	∞ / -
$k=1$	A ($d=0$)	Relaja B (180), Relaja C (450)	**0** / -	180 / A	450 / A	∞ / -	∞ / -	∞ / -
$k=2$	B ($d=180$)	Relaja D (400), Descarta C	**0** / -	**180** / A	450 / A	400 / B	∞ / -	∞ / -
$k=3$	D ($d=400$)	Relaja E (650), Descarta C	**0** / -	**180** / A	450 / A	**400** / B	650 / D	∞ / -
$k=4$	C ($d=450$)	Relaja F (800), Descarta E	**0** / -	**180** / A	**450** / A	**400** / B	650 / D	800 / C
$k=5$	E ($d=650$)	Descarta F	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	800 / C
$k=6$	F ($d=800$)	Sin aristas salientes	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	**800** / C

Nota. Notación *distancia / predecesor* ($d[v]/\pi[v]$). Cifras en negrita indican nodos cerrados en S.

Con $\pi = \{A : \text{NIL}, B : A, C : A, D : B, E : D, F : C\}$ se reconstruyen las rutas

$$A \to B \to D \to E: \quad 180 + 220 + 250 = 650 \text{ m}, \tag{12}$$

$$A \to C \to F: \quad 450 + 350 = 800 \text{ m}. \tag{13}$$

Al repetir el cálculo con la métrica temporal de la Tabla 6, las rutas óptimas coinciden y los costos son 165.33 s y 144.45 s, respectivamente. La coincidencia no es casual: en esta instancia las penalizaciones por pendiente no alteran el orden de los caminos. La Tabla 8 reporta el vector completo.

Tabla 8
Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor

Nodo	Nombre de la Intersección	Distancia Pura (m)	Tiempo Efectivo (s)	Secuencia de Ruta Óptima
A	Plaza Mayor del Cusco	0.0	0.0	`A`
B	Plaza Regocijo (Cusipata)	180.0	44.8	`A ->B`
C	Qorikancha (Santo Domingo)	450.0	108.0	`A ->C`
D	Plaza San Francisco	400.0	101.6	`A ->B ->D`
E	Mercado Central San Pedro	650.0	165.3	`A ->B ->D ->E`
F	Plazoleta Limacpampa	800.0	144.4	`A ->C ->F`

Nota. Solución calculada desde el vértice fuente $s =$ A (Plaza Mayor).

5.5. Instancia aplicada: red OSM de Cusco

La red real se obtuvo con OSMnx 2.1.1 alrededor del centro de Cusco $(-13.5165, -71.9788)$, con radio de 1.5 km y tipo `drive`. El *snapshot* contiene 891 vértices y 2139 aristas dirigidas; la razón $|E|/|V| = 2.40$ confirma el carácter disperso de la red. La consulta aplicada se eligió de forma determinista: origen en el vértice más cercano al centro de extracción y destino en el vértice alcanzable de mayor tiempo base. El camino óptimo tiene 17 vértices, 2810.6 m y un costo base de 387.197 s. Los detalles y la figura de la ruta se presentan en la Sección 10.

6. Diseño Algorítmico y Estructuras de Cola

6.1. Algoritmo de Dijkstra parametrizado

El Algoritmo 1 describe el núcleo común. La única dependencia de la cola es el contrato `insert`, `extract_min` y `decrease_key`; el resto del algoritmo no cambia.

Algorithm 1 Dijkstra con cola de prioridad parametrizada

Require: $G = (V, E, w)$, origen $s \in V$, cola Q
Ensure: $d : V \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, $\pi : V \to V \cup \{\text{NIL}\}$

for all $v \in V$ **do**
  $d[v] \leftarrow \infty$; $\pi[v] \leftarrow$ NIL
end for
$d[s] \leftarrow 0$; Q.insert$(s, 0)$; $S \leftarrow \emptyset$
while not Q.is_empty() **do**
  $(d_u, u) \leftarrow Q$.extract_min()
  if $d_u = \infty$ **then**
    break
  end if
  $S \leftarrow S \cup \{u\}$
  for all $(u, v) \in E$ **do**
    if $v \notin S$ **and** $d_u + w(u, v) < d[v]$ **then**
      $d[v] \leftarrow d_u + w(u, v)$; $\pi[v] \leftarrow u$
      if $v \in Q$ **then**
        Q.decrease_key$(v, d[v])$
      else
        Q.insert$(v, d[v])$
      end if
    end if
  end for
end while
return (d, π)

6.2. Invariante y corrección

Invariante. Al inicio de cada iteración del ciclo, para todo $u \in S$, $d[u] = \delta(s, u)$; para todo $v \in Q$, $d[v]$ es el costo mínimo de un camino de s a v con vértices intermedios en S.

Demostración. La base es $S = \emptyset$, $d[s] = 0$. Supóngase el invariante para $|S| = k$ y sea u el mínimo extraído. Si existiera un camino P de s a u con $w(P) < d[u]$, sea (x, y) la primera arista de P que sale de S. Entonces $x \in S$, $y \in Q$ y

$$w(P) \geq \delta(s, x) + w(x, y) = d[y] \geq d[u], \tag{14}$$

pues los pesos son no negativos y u minimiza Q. Contradicción. Por tanto $d[u] = \delta(s,u)$. Al terminar, $Q = \emptyset$ o solo quedan vértices inalcanzables, y el invariante cubre toda la componente alcanzable.

6.3. Heap binario indexado

El heap binario se almacena en un arreglo contiguo. Para que `decrease_key` no requiera una búsqueda lineal, se mantiene un mapa $\mathrm{pos}[v]$ con la posición actual de v. Cada intercambio actualiza el arreglo y el mapa:

$$\mathrm{heap}[i] \leftrightarrow \mathrm{heap}[j], \qquad \mathrm{pos}[\mathrm{heap}[i]] \leftarrow i, \qquad \mathrm{pos}[\mathrm{heap}[j]] \leftarrow j. \tag{15}$$

Así, `decrease_key` localiza el nodo en $O(1)$ y sube a lo sumo $\lfloor \log_2 |V| \rfloor$ niveles. El costo espacial es $O(|V|)$.

6.4. Heap 4-ario indexado

El heap 4-ario reduce la altura a $\lceil \log_4 |V| \rceil$. La aritmética de índices es la de la ecuación (4). La operación `sift-down` examina cuatro hijos por nivel; la operación `sift-up` hace una comparación por nivel. Con $d = 4$ fijo, el costo total de Dijkstra es

$$O\big((|E| + 4|V|) \log_4 |V|\big) = O\big((|E| + |V|) \log |V|\big), \tag{16}$$

con una constante que depende de d. La ventaja potencial no es asintótica, sino de localidad: los cuatro hijos de un nodo ocupan posiciones contiguas y suelen caber en una misma línea de caché (LaMarca & Ladner, 1996). Esa ventaja puede desaparecer si el tamaño es pequeño o si el costo de comparar cuatro hijos domina.

6.5. Fibonacci heap

El Fibonacci heap mantiene una lista circular de árboles. `insert` añade un nodo a la lista en $O(1)$; `decrease_key` corta el nodo si viola la propiedad de montículo y propaga el corte en cascada si el padre ya estaba marcado; `extract_min` promueve los hijos y consolida raíces de igual grado. Sus cotas amortizadas son $O(1)$ para `insert` y `decrease_key`, y $O(\log |V|)$ para `extract_min`. El costo total es $O(|E| + |V| \log |V|)$ (Fredman & Tarjan, 1987). La cota es mejor en el peor caso asintótico, pero no dice nada sobre las constantes de asignación de objetos, indirección de punteros y fallos de caché.

6.6. Complejidades comparadas

Tabla 9
Cotas temporales y espaciales de las colas evaluadas

Estructura	Insert	Extract-Min	Decrease-Key	Espacio	Costo total en Dijkstra
Arreglo no ordenado	$O(1)$	$O(\|V\|)$	$O(1)$	$O(\|V\|)$	$O(\|V\|^2 + \|E\|)$
Heap binario	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\|V\|)$	$O((\|V\| + \|E\|) \log \|V\|)$
Heap 4-ario	$O(\log_4 \|V\|)$	$O(4 \log_4 \|V\|)$	$O(\log_4 \|V\|)$	$O(\|V\|)$	$O((\|E\| + 4\|V\|) \log_4 \|V\|)$
Fibonacci	$O(1)$	$O(\log \|V\|)^*$	$O(1)^*$	$O(\|V\|)$	$O(\|E\| + \|V\| \log \|V\|)$

Nota. *Cota amortizada. Para d fijo, $\log_d |V| = O(\log |V|)$; la diferencia práctica está en las constantes y en la localidad de memoria.

6.7. Criterios de selección

La Tabla 10 resume la recomendación. No se declara un ganador universal: se indica el escenario en el que cada alternativa es razonable y la evidencia que lo respalda.

Tabla 10
Guía de selección según el escenario

Alternativa	Escenario recomendado	Justificación
Dijkstra + heap binario	Valor por defecto en redes pequeñas o medianas y cuando se prioriza simplicidad y robustez.	Menor constante de implementación; ganó la consulta OSM y 3 de 10 escalas sintéticas.
Dijkstra + heap 4-ario	Candidato en escalas densas o medianas, después de medir en el entorno objetivo.	Menor mediana en 7 de 10 escalas, con ventaja mediana de 4–5 %, pero IQR solapados.
Dijkstra + Fibonacci	Estudio teórico o escenarios con muchísimas `decrease_key` y bajo costo de asignación.	Nunca obtuvo la menor mediana en este estudio; 1.12–1.59 veces más lento que el binario.
A^* con heurística consistente	Consultas punto a punto con coordenadas y destino conocido.	Reduce expansiones sin perder optimalidad; la magnitud depende de la fuerza de h.
Bellman-Ford	Oráculo de validación o grafos con pesos negativos sin ciclos negativos.	$O(\|V\|\|E\|)$; no compite en tiempo, pero valida la solución.

6.8. Correspondencia entre pseudocódigo y código

La Tabla 11 vincula las etapas del Algoritmo 1 con la implementación en `python/src/algorithms/dijkstra.py`. La función `dijkstra_solver` recibe la instancia, el origen, el tipo de cola y un destino opcional; `reconstruct_path` reconstruye la ruta desde π.

Tabla 11
Correspondencia entre pseudocódigo y código

Etapa	**Pseudocódigo**	**Implementación**
Inicialización	$d[v] \leftarrow \infty$, $\pi[v] \leftarrow$ NIL, $d[s] \leftarrow 0$	`dist`, `prev` y `dist[s]=0.0`.
Construcción de Q	insertar cada vértice con $d[v]$	`create_queue(queue_type)` e inserción inicial.
Extracción	$(d_u, u) \leftarrow Q$.extract_min()	`d_u, u = heap.extract_min()`.
Relajación	$d[u] + w(u,v) < d[v]$	`candidate = d_u + weight`.
Actualización	`decrease_key` o `insert`	`heap.decrease_key(...)` si el vértice ya está en Q; si no, `insert`.
Salida	(d, π)	`distances`, `predecessors`, `paths` y telemetría.

7. Diseño e Implementación del Sistema

7.1. Arquitectura modular

El sistema se organiza en cuatro capas con dependencias descendentes (Figura 2). La capa de algoritmos no conoce Streamlit ni los archivos: recibe un `GraphInstance` y una cadena que identifica la cola. La capa de datos encapsula la carga, la conversión y la serialización. Esta separación permite probar el núcleo sin interfaz y sustituir la fuente de datos sin reescribir el solver.

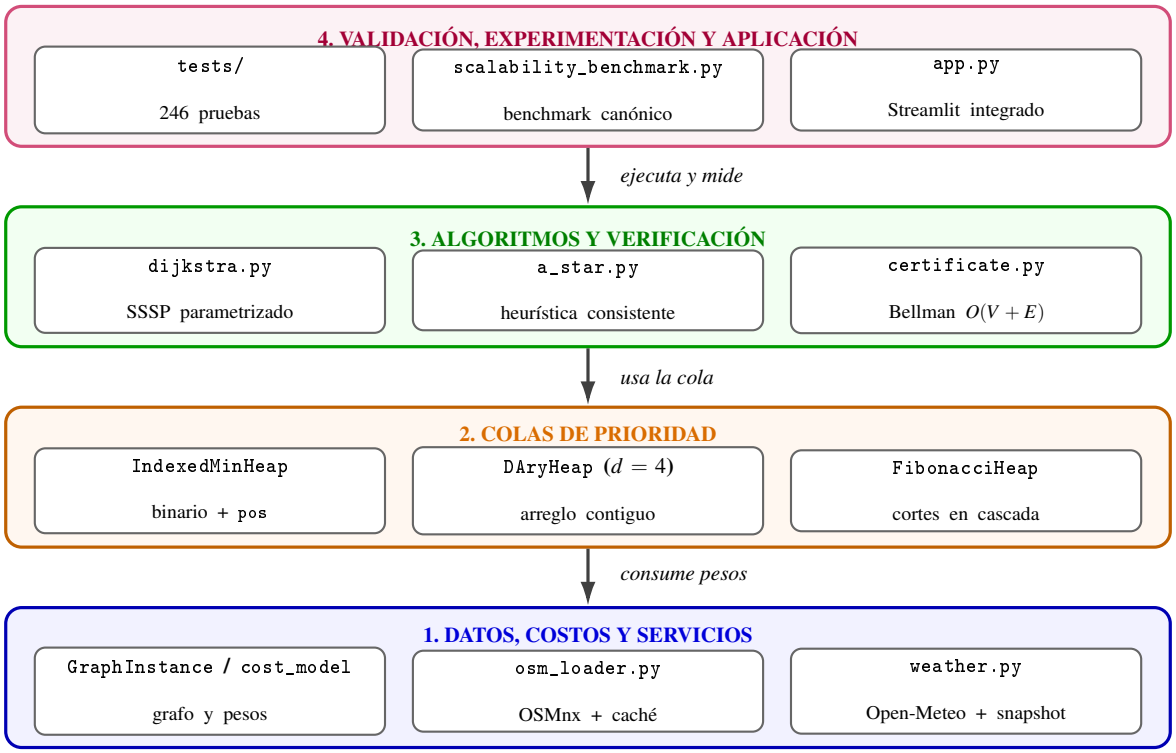


Figura 2
Arquitectura del sistema en cuatro capas

7.2. Módulos principales

1. `src/models/graph.py`: define `GraphInstance` (nodos, adyacencia, coordenadas, metadatos) y la carga/serialización.

2. `src/models/cusco_network.py`: construye la instancia ilustrativa de seis nodos y exporta su JSON.

3. `src/utils/cost_model.py`: calcula distancia o tiempo con la ecuación (11); valida longitud, métrica y factor climático.

4. `src/datastructures/`: implementa `IndexedMinHeap`, `DAryHeap` y `FibonacciHeap` con el contrato común.

5. `src/algorithms/dijkstra.py`: resuelve SSSP o una consulta punto a punto con `target`; registra operaciones y tiempos.

6. `src/algorithms/a_star.py`: calcula λ, construye h, valida consistencia y ejecuta A^*.
7. `src/algorithms/reference.py`: Bellman-Ford y Dijkstra con arreglo no ordenado como referencias.
8. `src/verification/certificate.py`: comprueba las condiciones de Bellman y compara alternativas.
9. `src/io/osm_loader.py`: descarga, proyecta, convierte y cachea la red OSM; `load_snapshot` funciona sin red.
10. `src/services/weather.py`: consulta Open-Meteo, guarda un *snapshot* y aplica un factor uniforme.
11. `src/benchmarks/scalability_benchmark.py`: genera instancias, ejecuta el protocolo canónico y escribe `benchmark_results.json`.
12. `src/benchmarks/applied_analysis.py`: ejecuta la consulta OSM, A^* y la sensibilidad climática.
13. `app.py`: aplicación Streamlit con las vistas Comparación, Ruta OSM, Traza manual y Procedencia.
14. `src/cli.py` y `main.py`: interfaz de comandos y pipeline de demostración.

7.3. Formatos de datos

Todos los formatos son JSON versionado y legible. La Tabla 12 resume los archivos principales.

Tabla 12
Archivos de datos y su función

Archivo	Contenido
data/instances/cusco_centro.json	Instancia ilustrativa: nodos, aristas, longitud, pendiente, tipo de vía y tiempo.
data/processed/cusco_osm_drive.json	Snapshot OSM: nodos con longitud/latitud, aristas dirigidas, longitud y tiempo, metadatos de extracción.
data/processed/benchmark_results.json	Protocolo, ambiente, muestras, medianas, IQR y operaciones por escala.
data/processed/applied_results.json	Consulta OSM, tiempos por cola, A^* y sensibilidad climática.
data/processed/weather_cusco.json	Snapshot meteorológico con hora, variables y fuente.

El esquema de la instancia ilustrativa es:

```
{
  "origin": "A", "destination": "B",
  "length_m": 180.0, "street_type": "calle_centro",
  "gradient_pct": 1.5, "street_name": "Calle Espaderos",
  "travel_time_sec": 44.82
}
```

Listing 1: Extracto del JSON de una arista

7.4. Contratos e inyección de colas

Las tres colas implementan:

- `insert(item, key)`: inserta un vértice con su clave tentativa.
- `extract_min()`: devuelve `(key, item)` y reestructura la cola.
- `decrease_key(item, new_key)`: actualiza una clave que solo puede disminuir.
- `is_empty()`: indica si quedan elementos.
- `op_stats()`: devuelve inserciones, extracciones, decrementos y comparaciones.

El solver recibe `queue_type` y llama a `create_queue`; no importa clases concretas. La misma inyección se usa en el benchmark y en la aplicación.

7.5. Manejo defensivo de entradas

La validación se concentra en `src/io/graph_io.py`: rechaza JSON mal formado, campos faltantes, identificadores duplicados, pesos negativos, NaN o infinito, y un origen inexistente. La CLI distingue tres códigos de salida: 0 (éxito), 1 (fallo de verificación) y 2 (entrada inválida). La aplicación muestra el error sin ejecutar el algoritmo. Las pruebas de `test_validation_io.py` cubren estos casos.

7.6. Decisiones técnicas

- **Sin `heapq`.** Las colas evaluadas se implementaron desde cero para que la comparación mida el diseño del proyecto y no una biblioteca en C.
- **Coordenadas opcionales.** Si una instancia no tiene coordenadas, A^* usa $h = 0$; no se aplican coordenadas de Cusco a grafos arbitrarios.
- **Snapshot antes que red.** La red OSM y el clima se guardan en disco; si la API falla, el sistema usa el archivo y lo indica en la interfaz.
- **Separación de resultados.** El benchmark principal no mezcla clima; la sensibilidad climática se reporta aparte para no atribuir al clima una diferencia entre colas.
- **Reproducibilidad.** Semillas fijas, versiones en `requirements.txt`, rutas relativas al paquete y comandos documentados en el Anexo B.

8. Estrategia de Pruebas y Validación Automatizada

8.1. Objetivo y organización

La validación persigue tres metas: (i) comprobar que las tres colas cumplen el mismo contrato; (ii) verificar que Dijkstra devuelve distancias óptimas y rutas reconstruibles; y (iii) asegurar que la carga de datos, la CLI, el servicio meteorológico y la aplicación funcionan sin red y rechazan entradas inválidas. La suite se ejecuta con `pytest` y se organiza por módulo, no por integrante, para que cualquier cambio pueda evaluarse de forma local.

```
python -m pytest python/tests -q
```

Listing 2: Ejecución de la suite completa

La Tabla 13 resume las 246 pruebas que componen la suite. El conteo incluye casos parametrizados: una misma función de prueba puede ejecutarse sobre las tres colas, varios tamaños o varias entradas inválidas. Una ejecución completa finalizó con `246 passed in 45.86s` en el entorno descrito en la Sección 9.

Tabla 13
Distribución de la suite automatizada

Archivo	Pruebas	Propósito principal
`test_priority_queues.py`	61	Contrato común, orden de extracción, claves inválidas, cortes en cascada y comparación con `heapq`.
`test_equivalence.py`	48	Equivalencia con Bellman-Ford, Dijkstra con arreglo y NetworkX; empates y escalas de 1000 y 3000 nodos.
`test_validation_io.py`	40	Validación de JSON, pesos negativos, NaN, campos faltantes, métricas y certificado.
`test_dijkstra_manual.py`	36	Traza manual de Cusco, CLRS 24.6, nodos inalcanzables, lazos, aristas paralelas y pesos extremos.
`test_cli.py`	25	Comandos, códigos de salida, generación, verificación, exportación y modo interactivo.
`test_dijkstra.py`	11	Casos básicos, límites y equivalencia a escala.
`test_a_star.py`	8	Optimalidad, consistencia, ausencia de coordenadas y regresión de heurística no admisible.
`test_benchmark_protocol.py`	4	Estadísticas robustas, perfiles y esquema del benchmark canónico.
`test_osm_loader.py`	4	Conversión determinista, métricas, caché y rechazo de snapshots inválidos.
`test_weather.py`	4	Multiplicador, clima adverso, *snapshot* y *fallback* sin red.
`test_applied_analysis.py`	3	Nodo más cercano, escalamiento uniforme y rechazo de factores inválidos.
`test_streamlit_app.py`	2	Carga de resultados canónicos y composición de la figura de ruta.
Total	**246**	

8.2. Patrón Arrange-Act-Assert

Cada prueba sigue tres fases explícitas:

- **Arrange.** Se construye la instancia, la cola o el archivo temporal, y se declara el resultado esperado.
- **Act.** Se invoca la operación bajo prueba: `insert`, `extract_min`, `decrease_key`, `dijkstra_solver` o un comando de la CLI.
- **Assert.** Se comprueba igualdad exacta o aproximada, orden, excepción esperada o código de salida.

Las comparaciones de distancias usan una tolerancia relativa de 10^{-6} para absorber el redondeo de punto flotante; las comparaciones de rutas verifican el costo, no la identidad de la secuencia, porque puede haber varios caminos óptimos.

8.3. Pruebas del contrato de colas

Las pruebas de `test_priority_queues.py` se parametrizan sobre las tres estructuras y cubren:

1. extracción en orden creciente y vaciado completo;
2. `decrease_key` hacia el frente, desde ∞ y sobre un elemento ausente;
3. rechazo de inserciones duplicadas y de claves NaN, cadenas, `None` o booleanos;
4. extracción sobre cola vacía, elemento único y claves repetidas;
5. comparación aleatoria contra una implementación de referencia, con seis variantes de heap y tres semillas;
6. ejercitación específica de cortes en cascada y consolidación del Fibonacci heap.

La comparación con `heapq` se usa solo como oráculo de orden, no como estructura del solver. Esto separa la corrección del contrato de la decisión de diseño de implementar las colas desde cero.

8.4. Oráculos de caminos mínimos

La corrección se contrasta con tres referencias independientes:

- **Bellman-Ford.** Tolerante a pesos negativos y capaz de detectar ciclos negativos; se usa en 40 grafos aleatorios de 5 a 122 nodos, con y sin pesos enteros, ceros y densidad $4|V|$.
- **Dijkstra con arreglo no ordenado.** Referencia $O(|V|^2 + |E|)$ sobre una red de 300 nodos; comprueba que la cola no altera las distancias.
- **NetworkX 3.7.** Oráculo externo sobre una cuadrícula dirigida de 8×8; evita que un error compartido por las tres implementaciones pase inadvertido.

Además, `test_dijkstra_manual.py` compara contra valores calculados a mano en la instancia de seis nodos y en la Figura 24.6 de Cormen et al. (2022). Esa prueba detecta errores de relajación que un oráculo automático podría reproducir si compartiera el mismo sesgo de implementación.

8.5. Certificado de optimalidad

Para pesos no negativos, un vector d es óptimo si y solo si cumple cuatro condiciones:

1. $d[s] = 0$;
2. para toda arista (u, v) con $d[u]$ finito, $d[v] \leq d[u] + w(u, v)$;
3. todo $v \neq s$ alcanzable tiene un predecesor u con $d[v] = d[u] + w(u, v)$;
4. cada camino reportado empieza en s, termina en v, usa aristas existentes y cuesta $d[v]$; los inalcanzables tienen $d[v] = \infty$ y camino vacío.

La verificación recorre cada arista y cada camino una vez, por lo que cuesta $O(|V| + |E|)$ y no vuelve a ejecutar Dijkstra. El certificado se aplica a las tres colas en cada instancia del benchmark y en la consulta OSM. `test_validation_io.py` incluye un caso con salida alterada para comprobar que el certificado efectivamente falla cuando debe fallar.

8.6. Pruebas de borde y regresión

La suite cubre los siguientes casos adversos:

- grafo de un solo nodo, origen sin aristas salientes y nodos inalcanzables;
- lazos, aristas paralelas, pesos cero, pesos muy grandes y muy pequeños, e identificadores enteros;
- cadenas largas sin recursión, para verificar que la reconstrucción de rutas es iterativa;
- ciclos negativos, que deben ser rechazados por Dijkstra y detectados por Bellman-Ford;
- archivos inexistentes, extensión incorrecta, JSON mal formado, codificación no UTF-8 y estructuras incompletas.

La prueba de regresión de A^* merece mención aparte. Una versión anterior aplicaba coordenadas de Cusco a cualquier grafo y producía una heurística no admisible. El caso de regresión construye un grafo con coordenadas que violan la cota y comprueba que la implementación actual rechaza la heurística o cae a $h = 0$; no se limita a comparar el resultado final.

8.7. Pruebas de integración sin red

Las pruebas de OSM y clima no dependen de Internet. `test_osm_loader.py` usa un *snapshot* temporal, comprueba que la conversión es determinista, que se preservan aristas paralelas y que la caché evita una segunda descarga. `test_weather.py` simula una respuesta, guarda el *snapshot* y verifica que `fetch_or_cached` usa el archivo cuando la red falla. La aplicación Streamlit se prueba con dos casos de humo: carga de los resultados canónicos y composición de la figura de ruta con tres capas (red, ruta y extremos).

8.8. Criterio de aceptación

Una versión se considera aceptable si: (i) las 246 pruebas pasan; (ii) el certificado de optimalidad se cumple en todas las instancias del benchmark; (iii) las tres colas producen las mismas distancias; y (iv) la aplicación y la CLI funcionan con el *snapshot* local aunque no haya red. No se reporta un porcentaje de cobertura porque no se midió con una herramienta de cobertura; afirmarlo sin medición sería una precisión ficticia.

9. Metodología Experimental y Diseño del Banco de Pruebas

9.1. Preguntas de investigación

El experimento responde tres preguntas:

1. **PI1.** ¿Qué cola minimiza el tiempo de pared de Dijkstra en redes viales sintéticas dispersas y densas, y cómo cambia la respuesta con la escala?
2. **PI2.** ¿La ventaja asintótica del Fibonacci heap se traduce en un menor tiempo de ejecución en una implementación de Python?
3. **PI3.** ¿El comportamiento observado en el banco sintético se reproduce en una consulta sobre una red vial real, y qué aportan la extensión A^* y el factor climático?

No se plantea una hipótesis de superioridad universal. La literatura experimental muestra que el resultado depende de la densidad, del tamaño y de la implementación (Cherkassky et al., 1996; LaMarca & Ladner, 1996); el objetivo es medir esa dependencia en el entorno local.

9.2. Hipótesis de trabajo

- **H1.** En redes dispersas, los heaps contiguos (binario y 4-ario) superarán al Fibonacci heap por menor sobrecarga de asignación y mejor localidad de memoria.
- **H2.** Al aumentar la densidad, el mayor número de operaciones `decrease_key` favorecerá relativamente al heap 4-ario y al Fibonacci heap.
- **H3.** A^* con una heurística consistente reducirá el número de expansiones sin alterar el costo óptimo.
- **H4.** Un factor climático uniforme incrementará el tiempo estimado, pero no cambiará la ruta óptima, porque multiplica todos los pesos por la misma constante.

H1 y H2 son hipótesis empíricas y pueden refutarse. H3 y H4 son propiedades que se verifican, no se presumen: la consistencia de h se comprueba antes de ejecutar A^* y la invariancia de la ruta se comprueba comparando las secuencias.

9.3. Diseño factorial

El benchmark canónico es un diseño factorial con tres factores:

- **Cola** (3 niveles): heap binario indexado, heap 4-ario indexado y Fibonacci heap.
- **Densidad** (2 niveles): dispersa, $|E| = 4|V|$; densa, $|E| = 0.08\,|V|(|V|-1)$.
- **Escala** (5 niveles por perfil): $|V| \in \{100, 500, 1000, 2500, 5000\}$ para el perfil disperso y $|V| \in \{50, 100, 200, 400, 800\}$ para el denso.

Cada combinación de densidad y escala usa tres instancias independientes. Cada instancia se resuelve con las tres colas y se mide 15 veces por cola, con dos ejecuciones de calentamiento. En total se obtienen

$$2 \text{ perfiles} \times 5 \text{ escalas} \times 3 \text{ instancias} \times 3 \text{ colas} \times 15 \text{ repeticiones} = 1350 \tag{17}$$

muestras cronometradas, además de 180 ejecuciones de calentamiento. La unidad experimental es la ejecución completa de Dijkstra desde el nodo `N0`; no se mide una operación aislada de la cola.

9.4. Generación de instancias

Las instancias se generan con `benchmark_network` y semilla fija. Para cada escala n y cada índice $j \in \{0,1,2\}$, la semilla es

$$s(n,j) = \begin{cases} 12345 + 31n + j, & \text{perfil disperso,} \\ 22345 + 31n + j, & \text{perfil denso.} \end{cases} \tag{18}$$

El generador construye primero una cadena dirigida $N_0 \to N_1 \to \cdots \to N_{n-1}$, que garantiza que todos los vértices sean alcanzables desde el origen, y luego añade aristas dirigidas aleatorias hasta alcanzar el número objetivo. Los pesos son uniformes en $[1, 1000]$ con tres decimales. La cadena evita el caso trivial de una componente desconectada y hace que el número de extracciones sea $|V|$ en todas las instancias.

El perfil denso usa $0.08\,|V|(|V|-1)$ aristas, es decir, el 8 % de las aristas posibles. No se usa el grafo completo porque su costo de memoria y de generación crece cuadráticamente y no representa una red vial. El perfil disperso usa $4|V|$, una razón cercana a la observada en la red OSM ($|E|/|V| = 2.40$).

9.5. Protocolo de medición

Cada medición sigue estos pasos:

1. Se construye la instancia y se ejecutan las tres colas una vez para verificar que las distancias coinciden con NetworkX y que el certificado de optimalidad se cumple.
2. Se baraja el orden de las colas con una semilla derivada de la instancia, para evitar que el orden de ejecución favorezca sistemáticamente a una estructura.
3. Para cada cola se ejecutan dos calentamientos sin cronometrar.
4. Se realizan 15 repeticiones. Antes de cada una se llama a `gc.collect()` y se desactiva el recolector; al terminar se restaura su estado. El tiempo se mide con `time.perf_counter_ns`.
5. Se registran mediana, cuartiles, IQR, MAD, mínimo, máximo, media, desviación estándar y coeficiente de variación.

La desactivación del recolector reduce la probabilidad de que una pausa de recolección caiga dentro de una muestra. No elimina el ruido del sistema operativo, pero lo hace más comparable entre colas. No se fija la afinidad de CPU ni se desactiva el *turbo boost*; ambas son fuentes de variación que se declaran en la Sección 11.

Estadística principal. Se reporta la mediana y el IQR. La media y la desviación estándar se conservan en el JSON, pero no se usan para declarar ganadores porque son sensibles a valores atípicos. Un menor IQR indica mayor estabilidad, no necesariamente menor tiempo. Cuando los IQR se solapan, la diferencia entre medianas no se interpreta como una ventaja concluyente.

Métricas de operaciones. Además del tiempo, se cuentan inserciones, extracciones, `decrease_key`, intentos de relajación y comparaciones. Estos contadores explican el comportamiento temporal, pero no sustituyen la medición: dos colas pueden hacer el mismo número de comparaciones y diferir por asignación de memoria o indirección.

Ausencia de métricas de memoria. No se midió el consumo de memoria residente ni el número de asignaciones. Por tanto, el informe no reporta RSS, *peak memory* ni comparaciones de espacio. Las cotas espaciales de la Tabla 9 son teóricas, no mediciones.

9.6. Caso aplicado sobre OpenStreetMap

La red aplicada se obtuvo con OSMnx 2.1.1 alrededor del centro de Cusco ($-13.5165, -71.9788$), con radio de 1.5 km y tipo `drive`. El *snapshot* contiene 891 vértices y 2139 aristas dirigidas. La consulta se selecciona de forma determinista: el origen es el vértice más cercano al centro de extracción y el destino es el vértice alcanzable con mayor tiempo base. Esta regla evita elegir el par a mano después de ver los resultados.

Para la consulta se ejecutan las tres colas con 30 repeticiones y tres calentamientos. El tiempo se mide con el mismo protocolo, pero el número de repeticiones es mayor porque la consulta es única y el costo de cada ejecución es pequeño. La corrección se verifica con el certificado y con la igualdad de distancias entre colas. La ruta se reconstruye y se compara con la obtenida por A^*.

Extensión A^*. Se usa la heurística escalada de la ecuación (6). Antes de ejecutar, se comprueba la consistencia en todas las aristas. Se comparan el costo óptimo y el número de expansiones de A^* y de Dijkstra punto a punto. La reducción de expansiones se reporta como un porcentaje; no se afirma una poda del 60–70 % porque esa cifra no se midió.

Sensibilidad climática. El factor climático se calcula con el *snapshot* de Open-Meteo y la función de la Sección 7. Se aplica un multiplicador uniforme a todos los pesos y se comparan el tiempo total y la secuencia de la ruta. El clima no se mezcla con el benchmark principal: se reporta como un escenario separado para no atribuir a la meteorología una diferencia entre colas.

9.7. Entorno de ejecución

La Tabla 14 resume el entorno. Los tiempos absolutos incluyen la sobrecarga del intérprete de Python y no son extrapolables a C++ o Rust. La comparación es válida entre las tres colas porque todas se ejecutan en el mismo proceso, con la misma instancia y el mismo intérprete.

Tabla 14
Entorno de ejecución del benchmark

Componente	Valor
Sistema operativo	Windows 11 (10.0.26300)
Procesador	AMD Ryzen 5 5600H, 6 núcleos, 12 hilos, 3.3 GHz
Memoria	24 GB
Python	3.12.8
pytest	9.1.1
NetworkX	3.7
OSMnx	2.1.1
Streamlit	1.64.0
Plotly	7.1.0

9.8. Reproducibilidad

El protocolo canónico, el caso aplicado, las tablas y las figuras se regeneran con un único script:

```
.\.venv\Scripts\python.exe python/scripts/reproduce_canonical.py
```

Listing 3: Regeneración completa desde la raíz

El script ejecuta 15 repeticiones por instancia, tres instancias por escala y semilla 12345; reutiliza los *snapshots* OSM y meteorológico versionados y escribe `benchmark_results.json`, `applied_results.json`, las tablas de `documento/tablas/` y las figuras de `documento/figuras/`. Para regenerar solo tablas y figuras a partir de los JSON existentes:

```
.\.venv\Scripts\python.exe python/scripts/reproduce_canonical.py --skip-benchmark --
    skip-applied
```

Listing 4: Regeneración de tablas y figuras sin repetir la medición

El subcomando `python main.py benchmark` conserva un modo exploratorio con escalas arbitrarias; no reemplaza al protocolo canónico. Las semillas, el número de repeticiones y el entorno quedan registrados en el JSON para que la comparación pueda repetirse o auditarse.

9.9. Amenazas a la validez

- **Validez interna.** El ruido del sistema operativo, el *turbo boost* y la migración de hilos pueden alterar muestras individuales. Se mitiga con calentamiento, GC desactivado, orden aleatorio y mediana/IQR, pero no se elimina.
- **Validez externa.** Los resultados corresponden a Python 3.12 en un Ryzen 5 5600H. Otra implementación, otro lenguaje u otra CPU pueden cambiar las constantes y el orden.
- **Validez de constructo.** El tiempo de pared incluye la reconstrucción de rutas y la telemetría del solver. Se desactivan la validación y la construcción de caminos durante el benchmark para reducir ese efecto, pero el costo de la cola no se mide de forma aislada.

- **Validez de conclusión.** Con 45 muestras por celda (tres instancias $\times$ 15 repeticiones) y distribuciones no normales, no se aplican pruebas de significancia. La conclusión se basa en medianas, IQR y consistencia entre escalas, no en un valor p.

10. Resultados Computacionales y Evaluación Experimental

10.1. Verificación de corrección

En todas las instancias del benchmark, las tres colas produjeron las mismas distancias y el certificado de optimalidad se cumplió. La comparación con NetworkX no detectó discrepancias y el certificado de Bellman recorrió todas las aristas y caminos sin violaciones. En la consulta OSM, las tres colas devolvieron la misma ruta y el mismo costo base de 387.197 s. La igualdad de distancias es la condición mínima para que la comparación de tiempos tenga sentido: si las colas resolvieran problemas distintos, el tiempo no sería comparable.

10.2. Benchmark sintético

Las Tablas 15 y 16 presentan la mediana y el IQR de 45 muestras por celda (tres instancias $\times$ 15 repeticiones). La Figura 3 muestra las curvas en escala logarítmica; las barras de error corresponden al IQR.

Tabla 15
Dijkstra con tres colas, perfil disperso: mediana e IQR en μs.

$\|V\|$	$\|E\|$	Binary Heap	4-ary Heap	Fibonacci Heap	Mejor
100	400	1648.4 [1377.9, 1979.3]	1673.3 [1393.8, 2186.5]	2574.3 [2371.2, 2870.8]	Binary
500	2000	19232.5 [14655.3, 24254.5]	14531.3 [11524.1, 18064.2]	30497.4 [20429.6, 38086.3]	4-ary
1000	4000	43378.1 [37780.5, 54962.6]	41803.4 [33976.3, 53954.7]	58348.7 [47548.7, 65855.4]	4-ary
2500	10000	105786.6 [96970.5, 121834.1]	106339.6 [94379.9, 122938.1]	138649.1 [121508.9, 170908.3]	Binary
5000	20000	254749.2 [219442.6, 316895.6]	212803.0 [199918.1, 232243.5]	285718.4 [261162.8, 315459.6]	4-ary

Nota. Cada celda muestra la mediana y, entre corchetes, el IQR.

Tabla 16

Dijkstra con tres colas, perfil denso: mediana e IQR en µs.

$\|V\|$	$\|E\|$	Binary Heap	4-ary Heap	Fibonacci Heap	Mejor
50	196	892.0 [821.0, 1074.6]	872.9 [783.2, 994.0]	1284.4 [1132.8, 1411.3]	4-ary
100	792	2287.0 [2057.4, 3227.4]	2182.2 [1894.3, 2405.6]	3142.8 [2655.2, 3976.6]	4-ary
200	3184	5138.6 [4040.9, 6430.3]	4852.3 [3881.0, 6065.4]	7169.3 [5476.0, 8327.4]	4-ary
400	12768	16408.9 [13090.2, 18599.3]	15487.6 [12208.9, 19884.7]	17183.7 [15487.0, 21265.9]	4-ary
800	51136	57923.5 [50208.7, 62974.9]	58345.2 [50691.3, 68398.6]	74614.7 [60800.1, 89075.6]	Binary

Nota. Cada celda muestra la mediana y, entre corchetes, el IQR.

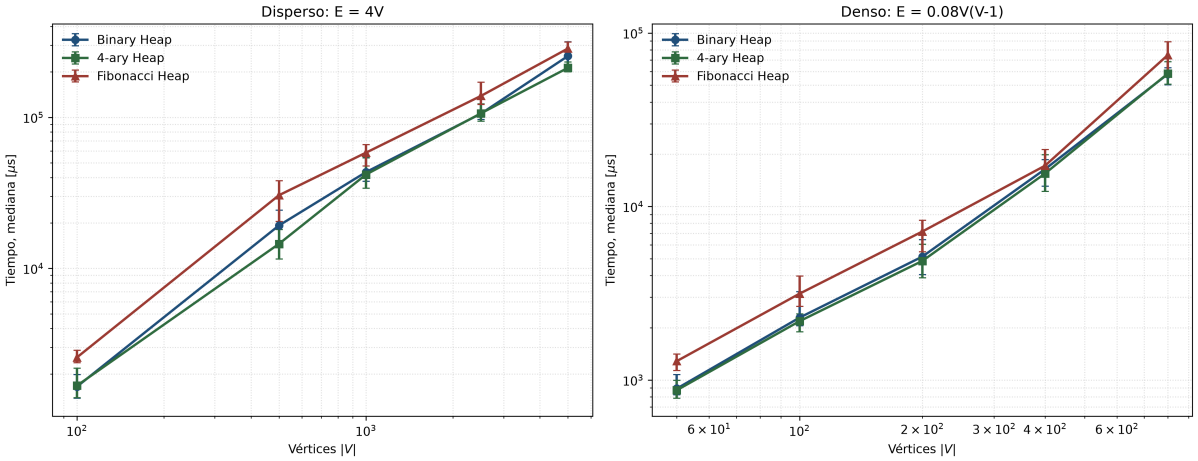


Figura 3

Tiempo mediano de Dijkstra con las tres colas. Izquierda: perfil disperso ($|E| = 4|V|$). Derecha: perfil denso ($|E| = 0.08|V|(|V|-1)$). Las barras verticales representan el IQR.

Perfil disperso. El heap 4-ario obtuvo la menor mediana en 3 de 5 escalas (500, 1000 y 5000), y el binario en 2 (100 y 2500). La razón mediana 4-ario/binario fue 0.964; es decir, el 4-ario fue un 3.6 % más rápido en la mediana de las escalas, pero la diferencia cambia de signo: fue 24.4 % más rápido en $|V| = 500$, 3.6 % en $|V| = 1000$, 16.5 % en $|V| = 5000$, y 1.5 % y 0.5 % más lento en $|V| = 100$ y $|V| = 2500$. Los IQR de ambas colas se solapan en las cinco escalas, por lo que la ventaja no es concluyente en ninguna escala individual.

Perfil denso. El 4-ario ganó 4 de 5 escalas (50, 100, 200 y 400) y el binario ganó en $|V| = 800$. La razón mediana fue 0.954 (4.6 % más rápido), con ventajas de 2.1 %, 4.6 %, 5.6 % y 5.6 % en las cuatro escalas ganadas, y una desventaja de 0.7 % en $|V| = 800$. Los IQR se solapan en las cinco escalas. La densidad no produjo un cambio cualitativo: el 4-ario conserva una ventaja pequeña y variable, no una superioridad sistemática.

Fibonacci heap. El Fibonacci heap no obtuvo la menor mediana en ninguna de las 10 escalas. Fue entre 1.05 y 1.59 veces más lento que el binario; en el perfil disperso, el rango fue 1.12–1.59, y en el denso, 1.05–1.44. Los IQR de Fibonacci y binario se solapan en 8 de 10 escalas; las dos excepciones son $|V| = 100$ disperso y $|V| = 50$ denso, donde el Fibonacci fue claramente más lento. La dirección del resultado es consistente: la cota asintótica no se tradujo en tiempo de pared.

Tabla 17

Síntesis comparativa por perfil y consulta. Las razones usan la mediana; no se declara un ganador cuando los IQR se solapan.

Perfil	4-ary/Binary	Fibonacci/Binary	Gana 4-ary	Gana Binary	IQR solap.
Disperso ($\|E\| = 4\|V\|$)	0.964	1.345	3 de 5	2 de 5	5 de 5
Denso ($\|E\| = 0.08\|V\|(\|V\| - 1)$)	0.954	1.374	4 de 5	1 de 5	5 de 5
Total sintético	0.959	1.360	7 de 10	3 de 10	10 de 10
Consulta OSM	1.269	1.666	0	1	–

Nota. Una razón 4-ary/Binary menor que 1 indica que el 4-ario fue más rápido. La columna IQR se refiere a 4-ary frente a Binary. Fibonacci no ganó ninguna escala sintética.

10.3. Operaciones elementales

La Figura 4 descompone las operaciones del perfil disperso. En $|V| = 5000$, las tres colas realizaron 5000 inserciones, 5000 extracciones y 6695 `decrease_key` efectivos sobre 10415 intentos de relajación. Los intentos que no mejoran la distancia no invocan `decrease_key`; por eso el número de decrementos es menor que $|E|$. En el perfil denso con $|V| = 800$, hubo 2976 decrementos sobre 25527 intentos, es decir, solo el 11.7 %.

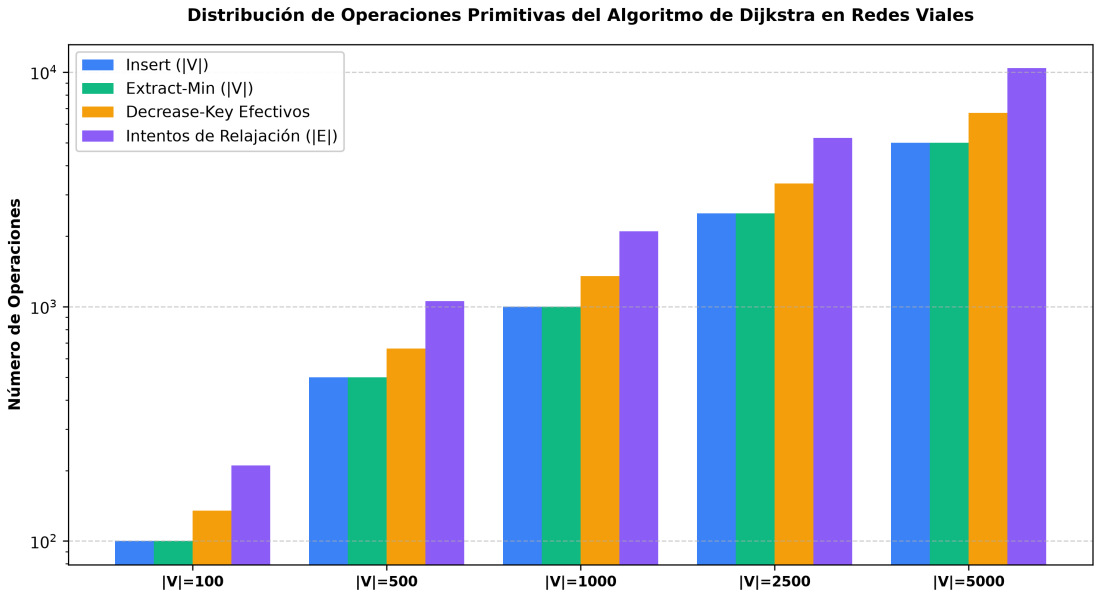


Figura 4

Operaciones primitivas de Dijkstra en el perfil disperso. Las barras agrupan inserciones, extracciones, `decrease_key` efectivos e intentos de relajación.

El conteo de comparaciones explica solo una parte del tiempo. En $|V| = 5000$ disperso, el

Fibonacci heap realizó 101303 comparaciones frente a 125776 del binario (19.5 % menos), pero fue 12.2 % más lento. En $|V| = 800$ denso, hizo 19848 comparaciones frente a 21280 (6.7 % menos) y fue 28.8 % más lento. La diferencia restante proviene de la manipulación de punteros, la asignación de nodos, los cortes en cascada y la consolidación, costos que no aparecen en el conteo de comparaciones. Esta observación es consistente con la literatura sobre localidad de memoria (LaMarca & Ladner, 1996).

10.4. Caso aplicado: consulta OSM en Cusco

La red aplicada tiene 891 vértices y 2139 aristas dirigidas. La consulta seleccionada por la regla determinista va del vértice `30972889` al `5996728243`; la ruta óptima tiene 17 vértices, 2810.6 m y un costo base de 387.197 s. La Figura 5 muestra la red, la ruta y los extremos.

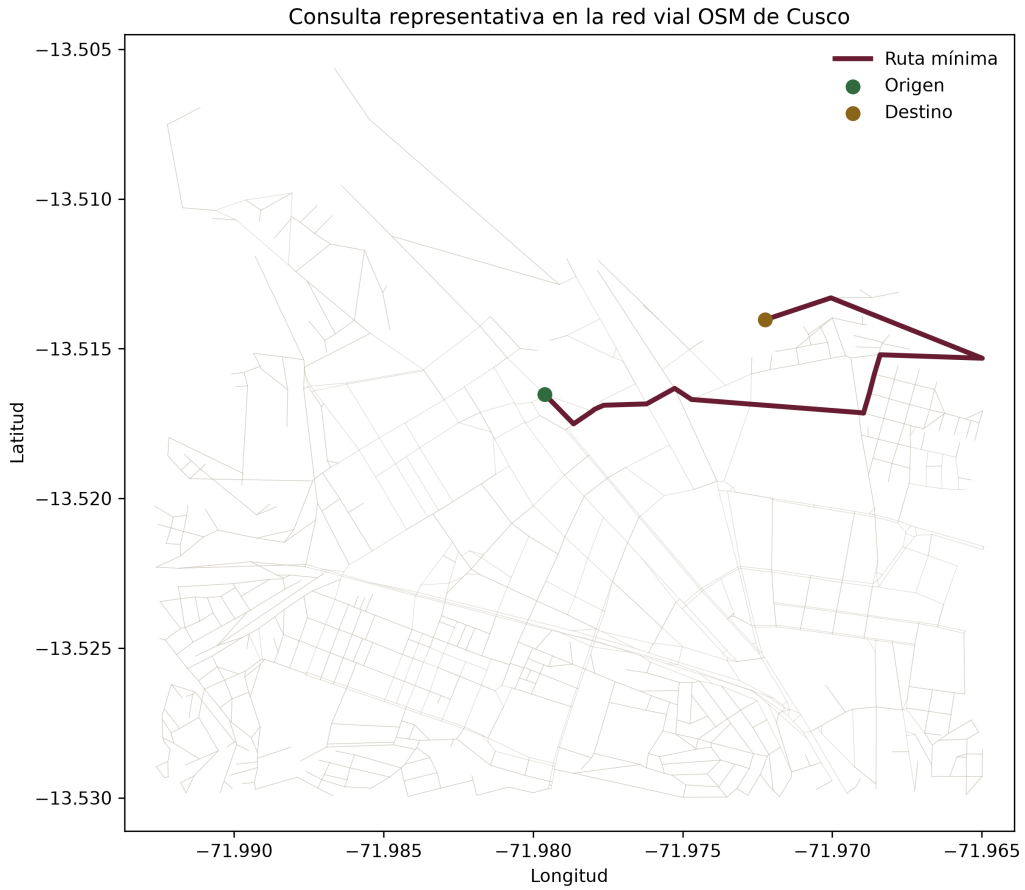


Figura 5
Ruta mínima sobre la red vial OSM de Cusco. La línea resalta el camino óptimo; los marcadores indican origen y destino.

La Tabla 18 reporta los tiempos de la consulta. El heap binario obtuvo la menor mediana (15.65 ms), seguido del 4-ario (19.87 ms, +26.9 %) y del Fibonacci (26.09 ms, +66.7 %). El IQR del binario [13.07, 19.55] se solapa con el del 4-ario [17.33, 22.68], de modo que la ventaja del binario frente al 4-ario es sugerente pero no concluyente; el IQR del Fibonacci [24.03, 27.34] no se solapa con el del binario, y su desventaja sí es clara en esta consulta.

Tabla 18
Caso OSM: tiempo de Dijkstra por cola, mediana e IQR.

Cola	Mediana (μs)	Q1 (μs)	Q3 (μs)
Binary Heap	15655.0	13065.6	19550.7
4-ary Heap	19872.2	17335.0	22675.6
Fibonacci Heap	26088.7	24033.9	27343.9

Tabla 19
Sensibilidad meteorológica uniforme en la consulta OSM.

Escenario	Tiempo (s)	Cambio (%)
Base	387.20	0.00
Snapshot meteorológico	413.95	6.91

Extensión A^*. Con la heurística escalada ($\lambda = 6484.687$), A^* expandió 823 vértices frente a 887 de Dijkstra punto a punto, una reducción de 64 expansiones (7.2 %). El costo óptimo fue idéntico. La reducción es modesta porque la heurística es una cota inferior geométrica y el destino está relativamente cerca en costo; no se observó la poda del 60–70 % que suele citarse en otros escenarios. El resultado confirma la propiedad teórica (mismo costo, menos expansiones) sin exagerar su magnitud.

10.5. Sensibilidad meteorológica

El *snapshot* de Open-Meteo registró 0.1 mm de precipitación, 8920 m de visibilidad, 40.7 km/h de ráfaga y 17.0 °C. Con los coeficientes de la Sección 7, el multiplicador fue

$$m_{\text{clima}} = 1 + 0.2\frac{0.1}{10} + 0.15\left(1 - \frac{8920}{10000}\right) + 0.1\frac{40.7}{80} = 1.0691. \tag{19}$$

El tiempo ajustado fue $387.197 \times 1.0691 = 413.952$ s, un incremento del 6.91 %. La ruta no cambió. Esto no es una coincidencia empírica: si todos los pesos se multiplican por la misma constante $m > 0$, entonces para todo camino P se cumple $w_m(P) = m\,w(P)$, y el camino que minimiza $w(P)$ también minimiza $w_m(P)$. El clima uniforme es, por construcción, un escenario de sensibilidad temporal; no puede alterar la elección de ruta. Para que el clima cambiara la ruta, el factor tendría que depender del tramo, por ejemplo de la pendiente, del tipo de vía o de la exposición al viento.

Síntesis de la evidencia. El 4-ary heap ganó 7 de 10 escalas sintéticas con IQR solapados; el binary heap ganó 3 y la consulta OSM; el Fibonacci no ganó ninguna y fue 1.05–1.59 veces más lento; A^* redujo un 7.2 % las expansiones sin cambiar el costo; y el clima incrementó el tiempo un 6.91 % sin alterar la ruta.

11. Discusión Crítica de Hallazgos

11.1. Teoría asintótica y tiempo de pared

La teoría ofrece una jerarquía clara. Para el Fibonacci heap,

$$T_{\text{Fib}} = O(|E| + |V| \log |V|), \tag{20}$$

mientras que para un heap binario indexado,

$$T_{\text{Bin}} = O((|V| + |E|) \log |V|). \tag{21}$$

La primera cota es menor cuando $|E|$ domina a $|V|$. Sin embargo, la notación $O(\cdot)$ oculta las constantes y el costo de las operaciones que no son comparaciones. En la implementación evaluada, el Fibonacci heap debe crear objetos, enlazar listas circulares, cortar nodos y consolidar raíces. Esas operaciones tienen un costo real que no aparece en (20). El resultado experimental —Fibonacci nunca gana y es hasta 1.59 veces más lento— no refuta la teoría: muestra que la cota asintótica describe el crecimiento, no el tiempo en una máquina y un lenguaje concretos.

La evidencia es consistente con Cherkassky et al. (1996) y LaMarca y Ladner (1996): en redes dispersas, los heaps contiguos suelen superar a las estructuras enlazadas porque sus ventajas de localidad y de asignación pesan más que la mejora amortizada de `decrease_key`. El Fibonacci heap sería competitivo si el número de decrementos fuera enorme frente al número de extracciones y si la asignación de nodos fuera barata. En las instancias medidas, los decrementos efectivos fueron 6695 frente a 5000 extracciones en $|V| = 5000$ disperso, y solo 2976 frente a 800 extracciones en $|V| = 800$ denso. La relación no es la de un régimen donde el término $|E|$ de (20) domine con holgura.

11.2. Por qué el heap 4-ario gana en algunas escalas

El heap 4-ario reduce la altura de $\lceil \log_2 |V| \rceil$ a $\lceil \log_4 |V| \rceil$. En $|V| = 5000$, la altura pasa de 13 a 7 niveles. Además, los cuatro hijos de un nodo ocupan posiciones contiguas y suelen caber en una misma línea de caché. Esa combinación explica por qué el 4-ario fue hasta 24.4 % más rápido en $|V| = 500$ y 16.5 % en $|V| = 5000$. Pero la ventaja no es gratuita: `sift-down` compara cuatro hijos por nivel y, en escalas pequeñas, el costo de esas comparaciones y de la aritmética de índices puede superar el ahorro de altura. Por eso el 4-ario fue ligeramente más lento en $|V| = 100$ y $|V| = 2500$ del perfil disperso, y en $|V| = 800$ del denso.

El hecho de que los IQR se solapen en las 10 escalas impide afirmar que el 4-ario sea mejor incluso donde su mediana es menor. El solapamiento no significa que las colas sean idénticas; significa que la variabilidad entre ejecuciones es del mismo orden que la diferencia entre medianas. Para declarar una ventaja estable harían falta más repeticiones, aislamiento de CPU o una carga de trabajo más larga. Con el diseño actual, la conclusión correcta es condicional: el 4-ario es un candidato, no un ganador universal.

11.3. Por qué el heap binario gana en la consulta OSM

En la consulta OSM, el binario fue 26.9 % más rápido que el 4-ario y 66.7 % más rápido que el Fibonacci. La red tiene solo 891 vértices y 2139 aristas, y la consulta punto a punto termina antes de explorar toda la componente. En ese régimen, el número de operaciones es pequeño y las constantes de implementación pesan más que la altura del árbol. El binario tiene la implementación más simple, menos comparaciones por nivel y menos aritmética de índices que el 4-ario; además, su IQR es amplio, lo que sugiere que el tiempo está dominado por el ruido del sistema y por el costo de relajación, no por la cola.

La comparación OSM no contradice el benchmark sintético: lo complementa. El 4-ario gana en escalas medianas y grandes del banco, pero pierde en una consulta pequeña y concreta. Esto refuerza la tesis de que la elección depende del tamaño, la densidad y el entorno. También advierte sobre un error frecuente: extrapolar el resultado de un benchmark sintético a una consulta real sin medirla.

11.4. El papel de la densidad

La densidad aumenta el número de intentos de relajación, pero no necesariamente el número de `decrease_key`. En el perfil denso con $|V| = 800$, hubo 25527 intentos y solo 2976 decrementos (11.7 %). La mayoría de los intentos no mejora la distancia tentativa y se descarta con una comparación. Por eso el Fibonacci heap, que optimiza `decrease_key`, tiene poco margen de mejora. El 4-ario, que optimiza la extracción y la localidad, sí puede aprovechar la mayor altura relativa del árbol. Aun así, la diferencia entre 4-ario y binario en el perfil denso fue solo del 4.6 % en la mediana, con IQR solapados.

11.5. Extensión A^*

A^* redujo las expansiones de 887 a 823 (7.2 %) sin alterar el costo. La reducción es menor que la que se obtiene cuando la heurística es muy informativa, porque $\lambda \|x_v - x_t\|$ es una cota inferior geométrica y no incorpora la estructura de la red ni los sentidos únicos. La heurística es consistente, y eso garantiza que no se reabran vértices cerrados; la optimalidad se mantiene. El resultado es útil como extensión, pero no convierte a A^* en el objeto principal del estudio: la comparación de colas se hizo con Dijkstra para aislar el efecto de la estructura.

11.6. Sensibilidad climática

El factor climático uniforme no puede cambiar la ruta. Si $w_m(e) = m\,w(e)$ para todo e y $m > 0$, entonces para dos caminos P y Q,

$$w_m(P) < w_m(Q) \iff m\,w(P) < m\,w(Q) \iff w(P) < w(Q). \tag{22}$$

La ruta óptima es la misma; solo cambia el costo total. El incremento del 6.91 % es, por tanto, una medida de sensibilidad del tiempo estimado, no un cambio de decisión. Para modelar un efecto real del clima sobre la ruta, el factor tendría que depender del tramo: por ejemplo, penalizar más las calles con pendiente, las vías sin pavimento o las expuestas al viento. Esa extensión queda fuera del alcance de este proyecto y se propone como trabajo futuro.

11.7. Marco de decisión

La evidencia permite formular una regla práctica:

1. **Valor por defecto: heap binario.** Es simple, robusto y ganó la consulta OSM y 3 de 10 escalas sintéticas. Para una aplicación nueva, es la opción con menor riesgo de implementación.
2. **Candidato: heap 4-ario.** Si un benchmark local, con la distribución real de consultas, muestra una ventaja consistente y los IQR no se solapan, el 4-ario puede adoptarse. No se justifica por la teoría sola.
3. **Fibonacci: uso teórico o experimental.** Su cota es valiosa para analizar el peor caso y para escenarios con muchísimos decrementos y asignación barata. En Python, no fue competitivo.
4. **A^*: consultas punto a punto con coordenadas.** Reduce expansiones sin perder optimalidad; su magnitud depende de la fuerza de la heurística.

11.8. Limitaciones

- **Una sola máquina y un solo lenguaje.** Los resultados corresponden a Python 3.12 en un Ryzen 5 5600H. En C++ o Rust, las constantes y posiblemente el orden cambiarían.
- **Sin métricas de memoria.** No se midió RSS ni asignaciones; las comparaciones espaciales son teóricas.
- **Sin pruebas de significancia.** Con distribuciones no normales y 45 muestras por celda, se usaron medianas e IQR. No se reportan valores p ni intervalos de confianza.
- **Instancias sintéticas.** El generador produce grafos aleatorios alcanzables, no redes viales geométricas. La red OSM mitiga parcialmente esta limitación, pero solo con una consulta.
- **Clima de modelo.** Open-Meteo entrega datos de modelo, no de una estación física; el factor es un escenario uniforme, no una calibración de tránsito.
- **Instancia ilustrativa.** La red de seis nodos tiene nombres reales, pero longitudes, pendientes y topología esquemáticas; no es un levantamiento geográfico.

11.9. Validez externa

La conclusión más robusta no es un ranking fijo, sino una metodología: medir la cola en la carga de trabajo objetivo, reportar dispersión y no extrapolar de un perfil a otro. El resultado puede cambiar con la implementación, la CPU o la distribución de consultas. Por eso el sistema deja el benchmark y los datos versionados: la decisión debe repetirse en el entorno de despliegue, no heredarse de este informe.

12. Conclusiones y Recomendaciones

12.1. Balance de objetivos

1. **OE1. Modelado.** Se formuló el grafo vial dirigido, la función de costo temporal con pendiente y el factor climático uniforme. La no negatividad de los pesos quedó garantizada por construcción, y la instancia ilustrativa se separó explícitamente del levantamiento geográfico.

2. **OE2. Implementación.** Se implementaron desde cero el heap binario indexado, el heap 4-ario indexado y el Fibonacci heap, con un contrato común que permite sustituirlos dentro de Dijkstra sin modificar el solver. También se implementaron A^* con heurística consistente, la referencia de Bellman-Ford y el certificado de optimalidad.

3. **OE3. Validación.** La suite automatizada reúne 246 pruebas y finalizó con `246 passed`. Las tres colas produjeron las mismas distancias en todas las instancias, el certificado de Bellman se cumplió y la comparación con NetworkX no detectó errores.

4. **OE4. Evaluación.** Se ejecutó un benchmark canónico de 1350 muestras cronometradas, una consulta sobre una red OSM real de 891 vértices y 2139 aristas, una extensión A^* y un análisis de sensibilidad climática.

12.2. Respuesta a las preguntas de investigación

PI1. ¿Qué cola minimiza el tiempo y cómo cambia con la escala? No hay una cola ganadora en todas las escalas. El heap 4-ario obtuvo la menor mediana en 7 de 10 escalas sintéticas, con una razón mediana de 0.964 en el perfil disperso y 0.954 en el denso. El heap binario ganó 3 escalas y la consulta OSM. Los IQR se solaparon en las 10 escalas sintéticas, de modo que la ventaja del 4-ario es pequeña y dependiente del tamaño. En la consulta OSM, el binario fue 26.9 % más rápido que el 4-ario y 66.7 % más rápido que el Fibonacci.

PI2. ¿La ventaja asintótica del Fibonacci heap se traduce en tiempo de pared? No en esta implementación. El Fibonacci heap no ganó ninguna escala y fue entre 1.05 y 1.59 veces más lento que el binario. Realizó menos comparaciones (19.5 % menos en $|V| = 5000$ disperso y 6.7 % menos en $|V| = 800$ denso), pero el costo de asignación, punteros, cortes y consolidación dominó el tiempo. La cota $O(|E| + |V| \log |V|)$ sigue siendo válida como análisis asintótico; no es una predicción de rendimiento en Python.

PI3. ¿Se reproduce el comportamiento en la red real y qué aportan A^* y el clima? La consulta OSM confirmó que la elección depende del tamaño: en una red pequeña, el binario fue el más rápido. A^* redujo las expansiones de 887 a 823 (7.2 %) sin cambiar el costo. El clima incrementó el tiempo estimado un 6.91 %, pero no cambió la ruta, porque un factor uniforme multiplica todos los pesos por la misma constante.

12.3. Conclusión principal

La conclusión central es que *la elección de cola es una decisión empírica y condicionada*. La teoría asintótica ordena las estructuras para el peor caso, pero no determina cuál es más rápida en una red vial concreta, en un lenguaje concreto y en una máquina concreta. En este estudio, el heap binario es la opción por defecto más robusta; el heap 4-ario es un candidato con ventaja pequeña en escalas medianas y grandes; el Fibonacci heap no es competitivo en Python. Esa conclusión no se apoya en una sola escala ni en una sola consulta, sino en el patrón conjunto de 10 escalas sintéticas y una consulta real.

12.4. Contribuciones

- Un núcleo algorítmico modular con tres colas intercambiables, A^* y referencias independientes.
- Un protocolo de medición reproducible con semillas, calentamiento, GC desactivado, mediana e IQR, y datos versionados.
- Un caso aplicado sobre OpenStreetMap con consulta determinista, ruta reconstruida y certificado de optimalidad.
- Una aplicación Streamlit integrada que permite comparar resultados, explorar la ruta, revisar la traza manual y consultar la procedencia de los datos.
- Un servicio meteorológico con *snapshot* y *fallback* sin red, separado del benchmark principal.
- Una suite de 246 pruebas que cubre contrato de colas, optimalidad, entradas inválidas, CLI, OSM, clima y aplicación.

12.5. Recomendaciones

Para la práctica.

1. Usar el heap binario como valor por defecto en una aplicación nueva, salvo que un benchmark local indique lo contrario.
2. Probar el heap 4-ario con la distribución real de consultas y exigir que la ventaja sea mayor que la variabilidad (IQR) antes de adoptarlo.
3. No adoptar el Fibonacci heap en Python por su cota teórica; reservarlo para análisis de peor caso o para entornos con asignación muy barata.
4. Usar A^* cuando haya coordenadas y destino conocido; verificar la consistencia de la heurística antes de confiar en la optimalidad.

Para el trabajo futuro.

1. Medir memoria residente y asignaciones para completar la comparación espacial.
2. Repetir el benchmark en C++ o Rust y en otra CPU para evaluar la portabilidad de las conclusiones.

3. Añadir pruebas de significancia con intervalos de confianza o *bootstrap*, y aumentar el número de instancias por escala.

4. Modelar el clima por tramo, con pendiente, tipo de vía y exposición, para que el factor pueda alterar la ruta.

5. Extender la consulta OSM a un conjunto de pares origen-destino y a distintas horas, para medir la variabilidad entre consultas.

6. Incorporar congestión dinámica y ventanas temporales, manteniendo la no negatividad de los pesos.

12.6. Cierre

El proyecto cumple los objetivos dentro de su alcance y deja explícitas sus limitaciones. La aportación más útil no es un ranking definitivo, sino un método para decidir con evidencia: implementar las alternativas, verificar la corrección, medir con dispersión y repetir la medición en el entorno de despliegue. Esa es la lección transferible a otros problemas de caminos mínimos y a otras estructuras de datos.

Referencias Bibliográficas

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126-139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

Cherkassky, B. V., Goldberg, A. V., & Radzik, T. (1996). Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, *73*(2), 129-174. https://doi.org/10.1007/BF02592101

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*(1), 269-271. https://doi.org/10.1007/BF01386390

Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, *34*(3), 596-615. https://doi.org/10.1145/28869.28874

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

LaMarca, A., & Ladner, R. E. (1996). The influence of caches on the performance of heaps. *ACM Journal of Experimental Algorithmics*, *1*, 4-es. https://doi.org/10.1145/235291.235295

Tarjan, R. E. (1983). *Data Structures and Network Algorithms*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9781611970265

Anexos

Anexo A: Instalación y configuración

El proyecto se ejecutó con Python 3.12.8 en Windows 11. Los comandos siguientes reproducen el entorno desde la raíz del repositorio:

```
git clone https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC.git
cd PROYECTO_Algoritmos-Avanzados-UNSAAC
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r python/requirements.txt
```

Listing 5: Clonación, entorno virtual e instalación

Los requisitos principales son Python 3.10 o superior, `pytest`, `networkx`, `osmnx`, `streamlit`, `plotly`, `pandas`, `numpy` y `matplotlib`. La compilación del informe requiere una distribución LaTeX con `pdflatex` y `biber`. La red OSM y el clima se versionan como *snapshots*; no es obligatorio descargarlos para ejecutar las pruebas.

Anexo B: Uso del prototipo

Suite de pruebas. Desde la raíz:

```
.\.venv\Scripts\python.exe -m pytest python/tests -q
```

Listing 6: Ejecución de las 246 pruebas

Interfaz de comandos. Desde `python/`:

```
..\.venv\Scripts\python.exe main.py list
..\.venv\Scripts\python.exe main.py run --queue all --trace
..\.venv\Scripts\python.exe main.py compare
..\.venv\Scripts\python.exe main.py astar --input data/processed/cusco_osm_drive.json
```

Listing 7: Comandos principales de la CLI

Aplicación web. Desde `python/`:

```
..\.venv\Scripts\python.exe -m streamlit run app.py
```

Listing 8: Ejecución de la aplicación Streamlit

La aplicación abre cuatro vistas: Comparación, Ruta OSM, Traza manual y Procedencia. Si no hay red, usa los *snapshots* locales y lo indica en pantalla.

Reproducción del benchmark y las figuras. Desde la raíz:

```
.\.venv\Scripts\python.exe python/scripts/reproduce_canonical.py
```

Listing 9: Regeneración de resultados, tablas y figuras

El script escribe `python/data/processed/benchmark_results.json`, `applied_results.json`, las tablas de `documento/tablas/` y las figuras de `documento/figuras/`. No sobrescribe la red OSM ni el clima si los *snapshots* ya existen.

Compilación del informe. Desde `documento/`:

```
.\compilar.ps1
```

Listing 10: Compilación con pdflatex y biber

Anexo C: Bitácora de aprendizaje autónomo

La Tabla 20 resume las dificultades técnicas y su resolución. Las fechas corresponden a las sesiones de trabajo del grupo.

Tabla 20
Bitácora de aprendizaje y resolución de dificultades

Integrante	Fecha	Fuente	Dificultad	Resolución
Choquenaira, Noe	18/09/2026	Arquitectura modular y Cormen et al. (cap. 24).	Desacoplar el solver de la cola y de la interfaz.	Definió las cuatro capas y el contrato `insert/extract_min/decrease_key`.
Choquenaira, Noe	21/09/2026	Modelos de impedancia vial.	Traducir pendiente y tipo de vía a un peso no negativo.	Formuló $w = (L/v)(1 + 2.5\,\text{máx}(0, \theta))m_{\text{clima}}$ y separó el clima del benchmark.
Choquenaira, Noe	02/10/2026	Heurísticas admisibles.	Evitar que A^* aplicara coordenadas de Cusco a grafos arbitrarios.	Introdujo $\lambda = \text{mín}\, w/\|\Delta x\|$, la verificación de consistencia y $h = 0$ sin coordenadas.
Porroa, Yeni	20/09/2026	Sedgewick y Wayne, colas indexadas.	La búsqueda lineal en `decrease_key` degradaba el costo.	Implementó la tabla inversa pos$[v]$ sincronizada en cada intercambio.
Porroa, Yeni	25/09/2026	LaMarca y Ladner (1996).	Aritmética de índices del heap 4-ario.	Implementó $\lfloor (i-1)/4 \rfloor$ y $4i + k$, y comparó la localidad con el binario.
Quispe, Romario	24/09/2026	Cormen et al. (cap. 19).	Enlaces circulares al extraer la raíz mínima.	Implementó `_remove_node` y `_add_root` con listas dobles.
Quispe, Romario	28/09/2026	Tarjan (1983), análisis amortizado.	Cortes en cascada con banderas `mark` inconsistentes.	Fijó la regla de desmarcar al ascender a raíz e interrumpir la cascada si el padre no estaba marcado.
Quispe, Romario	05/10/2026	Protocolos de medición.	Promedios inestables y ruido del recolector.	Adoptó `perf_counter_ns`, calentamiento, GC desactivado y mediana/IQR.
Yaranga, Aldo	29/09/2026	Patrón AAA y pytest.	Nodos inalcanzables y bucles en componentes disconexas.	Agregó corte temprano con distancia ∞ y pruebas de borde.
Yaranga, Aldo	03/10/2026	APA 7 y `booktabs`.	Tablas anchas fuera de margen.	Usó `tabularx`, tamaños relativos y exportadores LaTeX.
Yaranga, Aldo	12/10/2026	Streamlit y Plotly.	Reejecución del script y estado duplicado.	Separó funciones puras, cacheó datos y verificó escritorio/móvil.

Anexo D: Matriz de trazabilidad

La Tabla 21 vincula cada objetivo con su evidencia verificable.

Tabla 21
Trazabilidad entre objetivos, componentes y evidencia

Objetivo	Componente	Evidencia
OE1	Modelo de grafo y costo temporal	Sección 5; ecuación (11); instancia ilustrativa y red OSM.
OE2	Tres colas y Dijkstra parametrizado	Sección 6; `src/datastructures/`; 61 pruebas de contrato.
OE3	Validación y certificado	Sección 8; 246 pruebas; certificado $O(\|V\|+\|E\|)$; NetworkX y Bellman-Ford.
OE4	Benchmark, caso OSM, A^* y clima	Sección 10; 1350 muestras; 891 vértices; 823 vs 887 expansiones; factor 1.0691.
AG_C06.01	Aprendizaje autónomo	Sección 3; Anexo C.
AG_C06.02	Selección de stack	Sección 4; Tabla 4.
AG_C06.03	Experimentación y pensamiento crítico	Sección 9; Sección 11; conclusión condicionada.
Aplicación	Streamlit integrado	Sección 7; `app.py`; pruebas de humo.

Anexo E: Matriz de contribuciones

Tabla 22
Contribuciones por integrante y evidencia en el repositorio

Integrante	Componentes principales	Evidencia
Choquenaira, Noe	Arquitectura, modelo de costo, A^*, coordinación	`dijkstra.py`, `a_star.py`, `cost_model.py`, informe.
Porroa, Yeni	Heap binario y 4-ario indexados	`indexed_min_heap.py`, `d_ary_heap.py`, pruebas de colas.
Quispe, Romario	Fibonacci heap, benchmark y OSM	`fibonacci_heap.py`, `scalability_benchmark.py`, `osm_loader.py`.
Yaranga, Aldo	Pruebas, CLI, Streamlit y documentación	`tests/`, `cli.py`, `app.py`, exportadores LaTeX.

Anexo F: Evidencias de seguimiento

- **Sesión 1 (18/09/2026).** Definición del problema, arquitectura en cuatro capas y adopción de Python 3.12 y LaTeX.
- **Sesión 2 (24/09/2026).** Modelo de costo, instancia ilustrativa de seis nodos y primeras versiones de los heaps contiguos.
- **Sesión 3 (29/09/2026).** Integración del Fibonacci heap, suite AAA y certificado de optimalidad.
- **Sesión 4 (05/10/2026).** Protocolo canónico, red OSM, aplicación Streamlit y revisión de resultados.
- **Sesión 5 (28/10/2026).** Cierre del informe, verificación de tablas y figuras, y compilación del PDF.